

जावास्क्रिप्ट

सोप्या मराठीत

```
// JavaScript  
const learn = () => {  
  console.log("सुरुवात...");  
};  
  
let ideas = ["शिका",  
  "तयार करा",  
  "कुतीशील बना"];  
ideas.forEach((idea) => {  
  console.log(idea);  
});
```



BEGINNER-FRIENDLY JAVASCRIPT GUIDE

कोडिंग शिका • आत्मविश्वासाने तयार करा

नमस्कार मित्रांनो! 🙌



मी बोटू — तुमचा रोबोट मित्र!

या पुस्तकात आपण **JavaScript (जावास्क्रिप्ट)** ही भाषा शिकणार आहोत. ही भाषा वेबसाइट्सना **जिवंत** बनवते — बटण दाबले की काहीतरी घडते, घड्याळ चालते, चित्र बदलते! प्रत्येक गोष्ट मी चित्रांमधून आणि गोष्टींमधून सांगेन.

📖 हे पुस्तक कसे वापरायचे?

1. प्रत्येक धड्यात आधी एक छोटी गोष्ट किंवा उदाहरण असते — ते नीट वाचा.
2. प्रत्येक कल्पनेसाठी एक **चित्र** आहे. चित्र बघा, मग शब्द वाचा — लगेच समजेल!
3. काळ्या-निळ्या खोक्यात **कोड** आहे. तो स्वतः टाईप करून बघा. कॉपी करण्यापेक्षा टाईप केल्याने जास्त शिकायला मिळते.
4. शेवटी “**तुमची पाळी**” मध्ये एक छोटा प्रयोग आहे आणि “**लक्षात ठेवा**” मध्ये थोडक्यात उजळणी.

🔧 कोड कुठे लिहायचा आणि कसा चालवायचा?

1. कॉम्प्युटरवर **Notepad** (किंवा कोणताही साधा टेक्स्ट एडिटर) उघडा.
2. कोड लिहा आणि फाइल **नाव.html** अशा नावाने सेव्ह करा. उदा. **chapter_4.html**
3. त्या फाइलवर डबल-क्लिक करा — ती तुमच्या **ब्राउझरमध्ये** (Chrome, Edge, Firefox) उघडेल.
4. कोड बदलल्यावर सेव्ह करा आणि ब्राउझरमध्ये **Refresh (F5)** दाबा.



मोठ्यांसाठी एक टीप

हे पुस्तक “JavaScript for Beginners” या कोर्स-नोट्सचे **सोपे मराठी रूपांतर** आहे. शब्दशः भाषांतर न करता, लहान मुलांना समजेल अशी भाषा, गोष्टी आणि चित्रे वापरली आहेत. मूळ पुस्तकातील सर्व ३९ धडे येथे आहेत. मूळ पुस्तक जुन्या काळातले (Internet Explorer, **window.status** इ.) असल्याने काही ठिकाणी “**बोटू म्हणतो**” मध्ये आजच्या काळाची आठवण दिली आहे. कोड-उदाहरणे इंग्रजी अक्षरांतच ठेवली आहेत, कारण कॉम्प्युटर ती अशीच वाचतो.

अनुक्रमणिका

१	 प्रोग्रामिंग भाषा म्हणजे काय?	४	२१	 इव्हेंट हँडलर्स — घटनेला उत्तर	५६
२	 सर्वर-साइड आणि क्लायंट-साइड	६	२२	 चित्रांशी खेळ	५९
३	 JavaScript बदल थोडेसे	८	२३	 साधे इमेज रोलओव्हर	६२
४	 JavaScript ची सफर	१०	२४	 ऑब्जेक्ट तयार करा आणि चांगले रोलओव्हर	६४
५	 ऑब्जेक्ट्स, प्रॉपर्टीज आणि मेथड्स	१४	२५	 ब्राउझर विंडोज उघडणे	६७
६	 प्रॉपर्टीला किंमत देणे	१७	२६	 विंडो हव्या त्या जागी उघडणे	६९
७	 कमेंट्स — कोडमधल्या चिठ्ठ्या	२०	२७	 फोकस आणि ब्लर	७१
८	 जुन्या ब्राउझरपासून स्क्रिप्ट लपवणे	२२	२८	 वेळेवर बनणारे पान — Dynamic Content	७३
९	 वापरणाऱ्याला आपोआप दुसऱ्या पानावर नेणे	२४	२९	 अनेक विंडोज सांभाळणे	७५
१०	 Alert, Prompt आणि Confirm	२६	३०	 बाहेरची स्क्रिप्ट फाइल	७७
११	 व्हेरिअबल्स आणि ऑपरेटर्स	२८	३१	 JavaScript आणि फॉर्म्स	७९
१२	 तुलना	३१	३२	 फॉर्म मेथड्स आणि इव्हेंट हँडलर्स	८२
१३	 कंडिशनल्स — निर्णय घेणारा कोड	३३	३३	 JavaScript आणि गणित	८५
१४	 लूप्स — पुन्हा पुन्हा करा	३६	३४	 ऑब्जेक्ट व्हेरिअबल्स — उजळणी	८८
१५	 अॅर्रे — डब्यांची रांग	३९	३५	 मेनूमधून कृती	९०
१६	 नावाचे डबे — Associative Array	४२	३६	 फॉर्ममध्ये उत्तर आवश्यक करणे	९२
१७	 दोन मिती असलेले अॅर्रे	४४	३७	 तारखांशी काम	९५
१८	 मजकुराशी खेळ — स्ट्रिंग्स	४६	३८	 Date मधून माहिती काढणे	९७
१९	 फंक्शन्स वापरूया	५०	३९	 JavaScript चे घड्याळ बनवूया	१००
२०	 लॉजिकल ऑपरेटर्स — आणि / किंवा	५३			

१

धडा १

प्रोग्रामिंग भाषा म्हणजे काय?



कल्पना करा, तुमच्याकडे एक रोबोट मित्र आहे. तो खूप ताकदवान आहे, पण तो फक्त तुम्ही सांगितलेलेच करतो — त्यापेक्षा जास्त नाही, कमी नाही! कॉम्प्युटर अगदी तसाच असतो.

१ प्रोग्रामिंग भाषा म्हणजे कॉम्प्युटरशी बोलण्याची भाषा

आपण एकमेकांशी मराठीत बोलतो. कॉम्प्युटरशी बोलण्यासाठी एक खास भाषा लागते — तिला **प्रोग्रामिंग भाषा** म्हणतात. या भाषेत लिहिलेल्या सूचनांच्या यादीला **प्रोग्राम** (कोड) म्हणतात. कॉम्प्युटर त्या सूचना एकामागून एक पाळतो.



तुम्ही



सूचना (कोड)



कॉम्प्युटर



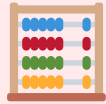
काम पूर्ण!

२ अनेक भाषा, पण सारख्या कल्पना

Java, C++, COBOL, BASIC, LISP अशा खूप प्रोग्रामिंग भाषा आहेत. त्यांतले शब्द बहुतेक इंग्रजीसारखे दिसतात. सगळ्या भाषांत काही गोष्टी सारख्याच असतात: **व्हेरिअबल** (माहितीचा डबा), **अॅर्रे** (डब्यांची रांग), **लूप** (पुन्हा पुन्हा करणे), **कंडिशन** (निर्णय घेणे) आणि **फंक्शन** (नाव दिलेले काम). म्हणजे एक भाषा शिकलात की दुसरी शिकणे सोपे जाते!



Java



C++



COBOL



BASIC



LISP



JavaScript



बोटू म्हणतो: या सगळ्या कल्पना आपण पुढच्या धड्यांत एकेक करून शिकणार आहोत. घाबरू नका!

३ प्रोग्रामचे दोन प्रकार

काही प्रोग्राम आपल्या कॉम्प्युटरमध्ये **इन्स्टॉल** करावे लागतात — जसे Microsoft Word किंवा Firefox. एकदा इन्स्टॉल केले की ते एकटेच चालतात. दुसरे प्रकार म्हणजे **वेब ॲप्लिकेशन** — हे ब्राउझरच्या आतच चालतात, जसे Google, Gmail आणि Flickr.

इन्स्टॉल करून चालणारे



Word



Firefox



Lotus Notes

VS

ब्राउझरमध्ये चालणारे



Google



Gmail



Flickr

४ वेब ॲप्सचे दोन भाग

वेब ॲप्लिकेशन अनेक तंत्रज्ञान एकत्र वापरून बनतात. त्यांचे सोपे दोन भाग करता येतात: **सर्व्हर-साइड** आणि **क्लायंट-साइड**. हे काय आहे ते पुढच्या धड्यात बघू.



तुमची पाली! करून बघा

1. एखादा गेम खेळताना, मोबाइलवर कॅल्क्युलेटर वापरताना किंवा Google वर शोधताना — तिथे “प्रोग्राम” आहे हे ओळखा.
2. तुमच्या आवडत्या दोन ॲप्सची नावे लिहा आणि ती इन्स्टॉल करून चालतात की ब्राउझरमध्ये ते ठरवा.

★ लक्षात ठेवा

- प्रोग्रामिंग भाषा म्हणजे कॉम्प्युटरला सूचना देण्याची भाषा.
- सगळ्या भाषांत व्हेरिअबल, ॲर्रे, लूप, कंडिशन आणि फंक्शन असतात.
- वेब ॲप्स ब्राउझरमध्ये चालतात.



हॉटेलमध्ये तुम्ही जेवायला गेलात. तुम्ही वेटरला ऑर्डर सांगता, वेटर किचनमधून जेवण आणतो. वेबवरही असेच घडते — तुम्ही = क्लायंट, किचन = सर्व्हर!

१ क्लायंट म्हणजे तुम्ही आणि तुमचा ब्राउझर

वेब पाहताना सुरुवात आणि शेवट तुमच्या **ब्राउझरपासूनच** होतो. तुम्ही, तुमचा कॉम्प्युटर आणि ब्राउझर — या सगळ्यांना मिळून **क्लायंट** म्हणतात. समजा तुम्ही www.google.com उघडले. तुमचा ब्राउझर Google च्या कॉम्प्युटरला पान मागतो. या मागणीला **रिक्वेस्ट** (request) म्हणतात.



क्लायंट (तुम्ही)



“Google चे पान द्या!”



सर्व्हर (Google)

२ सर्व्हर म्हणजे वेबसाइटचा कॉम्प्युटर

Google चा जो कॉम्प्युटर पाने सांभाळतो, त्याला **वेब सर्व्हर** म्हणतात. सर्व्हरचे काम सोपे आहे: मागितलेले पान शोधायचे आणि क्लायंटला परत पाठवायचे — म्हणजेच “सर्व्ह” करायचे, जसे वेटर जेवण वाढतो!



सर्व्हर



पान शोधतो



क्लायंटला पाठवतो



तुम्हाला दिसते!

३ सर्व्हर पान बदलू शकतो

PHP, Perl, Java यांसारख्या भाषा वापरून सर्व्हरला सांगता येते: “पान पाठवण्याआधी त्यात थोडा बदल कर.” याला **सर्व्हर-साइड ॲप्लिकेशन** म्हणतात. उदाहरण: पानावर **आत्ताची वेळ** लिहिणे. तुम्ही Refresh दाबता तेव्हा सर्व्हर नवीन वेळ घालून नवीन पान पाठवतो.



Refresh



प्रत्येक वेळी सर्व्हरकडून नवीन पान बनून येते

४ पान आल्यावर काय होते?

पान तुमच्या ब्राउझरवर आले की सर्व्हरचे काम संपले. आता सर्व्हर ते पान बदलू शकत नाही. पान बदलायचे असेल तर पुन्हा सर्व्हरकडे जावे लागते. पण आधुनिक ब्राउझर पानाच्या आतच **छोटे प्रोग्राम** चालवू शकतात. त्यांना **क्लायंट-साइड स्क्रिप्ट** म्हणतात. ते सर्व्हरकडे न जाता पान लगेच बदलू शकतात — उदाहरण: प्रत्येक सेकंदाला बदलणारे घड्याळ!



पान ब्राउझरमध्ये

+



छोटी स्क्रिप्ट

=



जिवंत घड्याळ!

५ दोघांची तुलना

दोन्हीचे फायदे आणि तोटे आहेत. क्लायंट-साइड स्क्रिप्ट वेगवान असतात, पण त्यांचा सगळा कोड वापरणाऱ्याकडे येत असल्याने तो कोणीही वाचू शकतो — म्हणून पासवर्ड तपासण्यासाठी त्या वापरू नयेत. सर्व्हरवरचा कोड गुप्त राहतो, पण पान बदलायला प्रत्येक वेळी नवीन पान आणावे लागते.

⚡ क्लायंट-साइड (स्क्रिप्ट)



लगेच प्रतिसाद



कोड सगळ्यांना दिसतो



पासवर्डसाठी योग्य नाही

VS

🏢 सर्व्हर-साइड



कोड गुप्त राहतो



सगळीकडे सारखे चालते



पान बदलायला नवीन पान लागते



बोटू म्हणतो: आपण या पुस्तकात **क्लायंट-साइड** म्हणजे ब्राउझरमध्ये चालणारी JavaScript शिकणार आहोत.



तुमची पाळी! करून बघा

- कुठल्याही वेबसाइटवर जा आणि Refresh दाबा. आता तुम्ही सर्व्हरकडे नवीन रिक्वेस्ट पाठवलीत!
- वहीत लिहा: हॉटेलच्या गोष्टीत क्लायंट कोण आणि सर्व्हर कोण?

★ लक्षात ठेवा

- क्लायंट = तुम्ही + तुमचा ब्राउझर. सर्व्हर = वेबसाइटचा कॉम्प्युटर.
- सर्व्हर पान पाठवतो; स्क्रिप्ट पान आल्यावर ते बदलते.
- JavaScript ही क्लायंट-साइड भाषा आहे.

३

धडा ३

JavaScript बद्दल थोडेसे



वेबसाइट म्हणजे एक घर आहे असे समजा. HTML म्हणजे भिंती, खिडक्या आणि दारे. पण दिवे चालू-बंद करणारे स्विच आणि वीज? ते आहे JavaScript!

१ JavaScript म्हणजे काय?

JavaScript ही वेबपानांना **जिवंतपणा** देणारी भाषा आहे. ती साध्या टेक्स्टमध्ये लिहिली जाते (Notepad मध्येही!). तिच्या मदतीने आपण पानावर बटणे, खेळ, घड्याळे आणि प्रश्नोत्तरे बनवू शकतो.



HTML (पान)

+



JavaScript (जादू)

=



जिवंत वेबसाइट

२ JavaScript काय काय करू शकते?

JavaScript ने आपण खूप मजेशीर गोष्टी करू शकतो: माउस बटणावर नेला की बटणाचे चित्र बदलणे, फॉर्ममध्ये लिहिलेली माहिती बरोबर आहे का ते तपासणे, नवीन विंडो उघडणे, वापरणाऱ्याचा ब्राउझर ओळखणे आणि सर्व्हरशिवाय पाने तयार करणे.



माउसने चित्र बदला



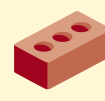
फॉर्म तपासा



नवीन विंडो उघडा



ब्राउझर ओळखा



पाने तयार करा

३ JavaScript आणि Java वेगळे आहेत!

नावात “Java” असले तरी **JavaScript आणि Java हे दोन पूर्णपणे वेगळे प्रोग्रामिंग** आहेत. जसे “पाव” आणि “पावसाळा” — नावात साम्य आहे, पण दोघांचा काहीही संबंध नाही! Java ही मोठी, ताकदवान भाषा आहे. JavaScript ही वेबपानांसाठी छोटी आणि सोपी भाषा आहे.

Java



मोठी, पूर्ण भाषा



C आणि C++ सारखी

VS

JavaScript



वेबपानांसाठी



ब्राउझरच्या आत चालते

४ चार खास शब्द — सोप्या भाषेत

JavaScript ची ओळख चार शब्दांत होते: **Interpreted** — कोड ब्राउझरमध्ये आला की एकेक ओळ लगेच चालते, जसे दुभाषी बोललेले लगेच भाषांतरित करतो. **Client-side** — ती तुमच्या ब्राउझरमध्ये चालते. **Event-based** — क्लिक किंवा माउस हलणे अशी घटना घडली की ठरावीक कोड चालतो. **Object-oriented** — पानातील प्रत्येक गोष्ट (चित्र, बटण, विंडो) एक **ऑब्जेक्ट** असते आणि आपण त्याला सूचना देतो.



Interpreted — लगेच चालते



Client-side — ब्राउझरमध्ये



Event-based — घटना घडली की



Object-oriented — वस्तूंना सूचना

५ HTML ही प्रोग्रामिंग भाषा नाही

HTML ही **मार्कअप भाषा** आहे — ती पानाला आकार देते (शीर्षक, परिच्छेद, चित्र, फॉर्म), पण ती विचार करू शकत नाही आणि हिशोब करू शकत नाही. म्हणून JavaScript नेहमी HTML बरोबर लिहितात, पण दोघांचे नियम वेगवेगळे आहेत.



HTML



पानावर खुणा करते



विचार/हिशोब करू शकत नाही

VS



JavaScript



विचार करते



हिशोब करते



निर्णय घेते



तुमची पाली! करून बघा

- वहीत लिहा: HTML आणि JavaScript मध्ये काय फरक आहे? घराच्या उदाहरणाने सांगा.
- तुमच्या आवडत्या वेबसाइटवर बटण दाबल्यावर काय घडते ते बघा — हे कदाचित JavaScript चे काम असेल!

★ लक्षात ठेवा

- JavaScript वेबपानांना जिवंत करते.
- Java आणि JavaScript पूर्णपणे वेगळे आहेत.
- HTML पानाचा सांगाडा, JavaScript त्याचा मेंदू.



धडा ४

JavaScript ची सफर



चला, एक छोटी सफर करूया! या धड्यात आपण JavaScript काय काय करू शकते ते फक्त बघणार आहोत. सगळे समजले नाही तरी चालेल — फक्त मजा करा!

१ पहिले पान तयार करा

Notepad उघडा, खाली दिलेला कोड टाईप करा आणि फाइल `chapter_4.html` या नावाने सेव्ह करा. मग त्या फाइलवर डबल-क्लिक करून ब्राउझरमध्ये बघा. हे फक्त HTML आहे — अजून JavaScript नाही.

chapter_4.html

Chapter 4: A Tour of JavaScript

A Tour of JavaScript

कोड

```
<html>
<head>
  <title>Chapter 4: A Tour of JavaScript</title>
</head>
<body>
  <h1>A Tour of JavaScript</h1>
</body>
</html>
```

२ JavaScript साठी खास जागा — script टॅग

JavaScript लिहिण्यासाठी आपण **<script>** नावाचा टॅग वापरतो. हा टॅग एखाद्या **खोक्यासारखा** आहे — आपला सगळा JavaScript कोड त्या खोक्याच्या आत लिहायचा. आपण तो सहसा **<head>** मध्ये ठेवतो, म्हणजे सगळा कोड एकाच ठिकाणी सापडतो.



कोड

```
<head>
  <title>Chapter 4</title>
  <script type="text/javascript">
    // इथे आपला JavaScript कोड येईल
  </script>
</head>
```



बोटू म्हणतो: जुन्या पुस्तकात `language="JavaScript"` असे लिहिले आहे. आजकाल फक्त **<script>** लिहिले तरी चालते.

३ पहिला कोड: Hello world!

alert म्हणजे एक छोटी **सूचना-पेटी** स्क्रीनवर दाखवणे. script च्या खोक्यात ही ओळ लिहा आणि पान Refresh करा. अभिनंदन — तुम्ही JavaScript च्या जगात पाऊल ठेवले! 🎉

या पानाचा संदेश

Hello world!

OK

कोड

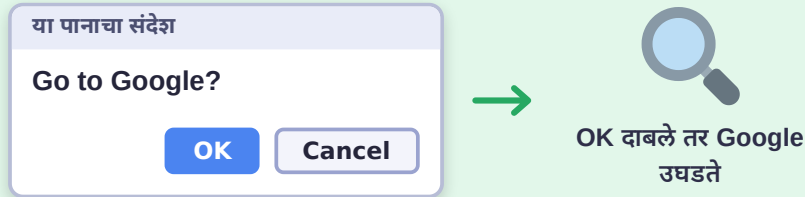
```
window.alert("Hello world!");
```



निकाल: ब्राउझरमध्ये "Hello world!" असा संदेश असलेली पेटी येते.

४ निर्णय घेणारा कोड: confirm

`confirm` एक प्रश्न विचारते: “OK” किंवा “Cancel”. जर तुम्ही OK दाबले, तर `if` (जर) च्या आतले काम होते — पान Google वर जाते. अशा “जर... तर...” ला **कंडिशनल** म्हणतात. म्हणजे JavaScript आपल्या उत्तरानुसार **निर्णय** घेते!

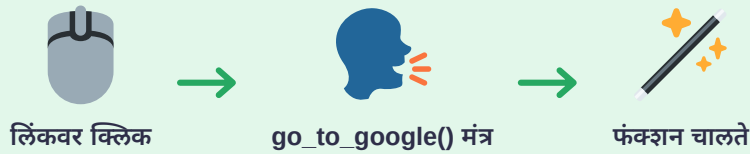


कोड

```
if ( confirm("Go to Google?" ) ) {  
  document.location = "http://www.google.com/";  
}
```

५ फंक्शन — जादूचा मंत्र आणि इव्हेंट

वरचे कोड पान उघडताच लगेच चालतात. पण आपल्याला ते **हवे तेव्हाच** चालवायचे असते. त्यासाठी कोडला नाव देऊन **फंक्शन** बनवतात. फंक्शन म्हणजे **जादूचा मंत्र** — ते नाव घेतले तरच जादू होते! आणि तो मंत्र म्हणायला लावणारी घटना म्हणजे **इव्हेंट**. खाली `onclick` म्हणजे “क्लिक केल्यावर” — लिंकवर क्लिक केले की ब्राउझर `go_to_google()` हा मंत्र म्हणतो.



कोड

```
function go_to_google() {  
  if ( confirm("Go to Google?" ) ) {  
    document.location = "http://www.google.com/";  
  }  
}
```

```
<a href="#" onclick="go_to_google();" >test</a>
```

६ JavaScript ने पानावर नवीन मजकूर लिहा

`document.write` पानावर नवीन HTML लिहू शकते. खालच्या कोडने पानावर नवीन शीर्षक दिसते. आत्ता हे HTML ने पण करता आले असते, पण पुढे आपण याने आजची तारीख, वेळ अशा गोष्टीही लिहिणार आहोत.

chapter_4.html

Here's another header!

कोड

```
<script type="text/javascript">
  document.write("<h2>Here's another header!</h2>");
</script>
```



बोटू म्हणतो: कोडची एक ओळ एकाच ओळीत लिहा. मध्येच नवीन ओळ घेतली तर कधीकधी कोड चालत नाही!



तुमची पाळी! करून बघा

1. वरचे सगळे कोड एकेक करून तुमच्या `chapter_4.html` मध्ये टाईप करा आणि प्रत्येक वेळी Refresh करा.
2. `alert` मधला "Hello world!" बदलून स्वतःचे नाव लिहा.

★ लक्षात ठेवा

- JavaScript कोड `<script>` च्या खोक्यात लिहितात.
- फंक्शन म्हणजे नाव दिलेला कोडचा गट — नाव घेतले तरच चालतो.
- इव्हेंट (उदा. `onclick`) फंक्शनला जागे करतो.



धडा ५

ऑब्जेक्ट्स, प्रॉपर्टीज आणि मेथड्स



तुमच्या घरात एक पियानो आहे असे समजा. पियानो एक वस्तू आहे. त्याचा रंग, वजन हे त्याचे गुण आहेत. आणि पियानो वाजवता येतो — ही क्रिया आहे. JavaScript मध्येही अगदी असेच असते!

१ ऑब्जेक्ट म्हणजे “वस्तू”

सोप्या भाषेत ऑब्जेक्ट म्हणजे एक वस्तू. आपल्या पियानोसारखीच. बोलण्यातली भाषा आठवा: वस्तू म्हणजे **नाम**, तिचे गुण म्हणजे **विशेषणे** आणि तिच्या क्रिया म्हणजे **क्रियापदे**.



piano = ऑब्जेक्ट (नाम)

+



property (विशेषण)

+



method (क्रियापद)

२ प्रॉपर्टी म्हणजे गुणधर्म

प्रॉपर्टी म्हणजे वस्तूचे वर्णन करणारे गुण. पियानोचा रंग, वजन, उंची, पेडल्स, कीबोर्ड आणि झाकण. गंमत अशी: पियानोचे **झाकण** ही पियानोची प्रॉपर्टी आहे, पण झाकणाचीही स्वतःची प्रॉपर्टीज आहेत — रंग, लांबी, आणि उघडे आहे की बंद!



piano

गुणधर्म (Properties) — विशेषणे

- colour (रंग)
- weight (वजन)
- height (उंची)
- pedals (पेडल्स)
- lid (झाकण — तेही एक ऑब्जेक्ट!)

क्रिया (Methods) — क्रियापदे

- play() — वाजवा
- lid.open() — झाकण उघडा
- pedals.sustain.press() — पेडल दाबा

३ मेथड म्हणजे क्रिया

मेथड म्हणजे वस्तू करू शकते किंवा आपण वस्तूवर करू शकतो अशी **कृती**. पियानो वाजवणे, झाकण उघडणे, पेडल दाबणे — ही सगळी मेथड्स आहेत.



piano



वाजवा (play)



बोट्ट म्हणतो: मेथडच्या नावानंतर नेहमी () कंस असतात — जसे `play()`.

४ डॉट (.) चा नियम

JavaScript मध्ये ऑब्जेक्ट, प्रॉपर्टी आणि मेथड एकमेकांना **टिंब (.)** ने जोडतात. म्हणजे “च्या”! `piano.colour` म्हणजे “पियानोचा रंग”. `piano.lid.open()` म्हणजे “पियानोचे झाकण उघडा”.

piano

.

lid

.

open()

कोड

```
piano.colour;           // पियानोचा रंग
piano.play();           // पियानो वाजवा
piano.lid.open();       // पियानोचे झाकण उघडा
piano.pedals.sustain.press(); // सस्टेन पेडल दाबा
```

५ खऱ्या वेबपानातील ऑब्जेक्ट्स

वेबपानावरची प्रत्येक गोष्ट JavaScript ला ऑब्जेक्ट म्हणून दिसते. आपण आधी वापरलेले `document.write(...)` बघा: **document** हा ऑब्जेक्ट (पान), **write** ही मेथड (लिहा). आणि `document.location` मध्ये **location** ही प्रॉपर्टी (पत्ता) आहे. कंस () असले की ती मेथड, नसले की प्रॉपर्टी.

कोड

`document.write("...")`

ऑब्जेक्ट

document

काय आहे?

write = मेथड (लिहिण्याची क्रिया)

`document.location`

document

location = प्रॉपर्टी (पानाचा पत्ता)

६ कंसात माहिती देणे (arguments)

मेथडला काम करताना काही माहिती हवी असते. ती कंसात लिहितात — याला **आर्ग्युमेंट** म्हणतात. `document.write("Hello World")` मध्ये “Hello World” हे आर्ग्युमेंट आहे. जास्त आर्ग्युमेंट असतील तर त्यांच्यामध्ये **स्वल्पविराम (,)** ठेवतात.



write



“Hello World”



पानावर Hello World

७ स्टेटमेंट आणि अर्धविराम (;)

JavaScript ची प्रत्येक सूचना म्हणजे एक **स्टेटमेंट**. जसे मराठी वाक्याच्या शेवटी पूर्णविराम असतो, तसे प्रत्येक स्टेटमेंटच्या शेवटी **अर्धविराम (;)** लिहावा. नाही लिहिला तर कधीकधी JavaScript चुकीचा अंदाज बांधते आणि कोड बिघडतो.

```
document.write("Hi" ;
```

कोड

```
document.write("Hello!"); // शेवटी ; विसरू नका!
```

तुमची पाली! करून बघा

1. तुमच्या खोलीतील एक वस्तू निवडा (उदा. पंखा). तिच्या दोन प्रॉपर्टीज आणि दोन मेथड्स वहीत लिहा.
2. त्या वस्तूसाठी डॉटवाले कोड लिहा, जसे `fan.speed` आणि `fan.start()`.

★ लक्षात ठेवा

- ऑब्जेक्ट = वस्तू (नाम), प्रॉपर्टी = गुण (विशेषण), मेथड = क्रिया (क्रियापद).
- डॉट (.) म्हणजे "च्या".
- मेथडला कंस () असतात; प्रत्येक स्टेटमेंट ; ने संपते.



धडा ६

प्रॉपर्टीला किंमत देणे



तुम्हाला तुमचा पियानो हिरवा रंगवायचा आहे. JavaScript मध्ये हे फक्त एका ओळीत सांगता येते!

१ थेट रंग बदला

प्रॉपर्टी बदलण्याचा सर्वात सोपा मार्ग म्हणजे तिला नवीन किंमत देणे. आपण `piano.paint("green")` असे मेथडने पण लिहू शकलो असतो, पण ती मेथड असेलच याची खात्री नसते. म्हणून थेट सांगतो: `piano.colour = "green";`



piano.colour



"green"



हिरवा पियानो

कोड

```
piano.colour = "green";
```

२ = चा खरा अर्थ

इथे = चा अर्थ "समान आहे" असा नाही, तर "ठेवा" असा आहे! उजवीकडची किंमत उचलून डावीकडच्या डब्यात ठेवा. याला **असाइनमेंट ऑपरेटर** म्हणतात.

piano.colour

green

आधी: brown → आता: green



बोद्ध म्हणतो: = म्हणजे "ठेवा". बरोबरी तपासायची असेल तर वेगळे चिन्ह (==) असते — ते आपण धडा १२ मध्ये बघू.

३ पानाचे शीर्षक बदला

ब्राउझरच्या वरच्या पट्टीत जे नाव दिसते ते म्हणजे पानाचे **title**. आपण ते JavaScript ने बदलू शकतो!



chapter_6.html

a new title

पान उघडले आहे...

कोड

```
document.title = "a new title";
```

४ असाइनमेंट ऑपरेटरचे भाऊ-बहिणी

“ठेवा” (=) शिवाय आणखीही काही असाइनमेंट ऑपरेटर आहेत. ते हिशोब आणि ठेवणे एकत्र करतात.

चिन्ह	अर्थ
$x = y$	x मध्ये y ची किंमत ठेवा
$x += y$	x मध्ये y मिळवा ($x = x + y$)
$x -= y$	x मधून y वजा करा
$x *= y$	x ला y ने गुणा
$x /= y$	x ला y ने भागा

५ += मजकुरालाही जोडते

`+=` संख्यांना बेरीज करते आणि मजकुराला (स्ट्रिंगला) जोडते. त्यामुळे `document.title += "!"` म्हटले की शीर्षकाच्या शेवटी “!” लागतो.

"JavaScript" + "!" = "JavaScript!"

कोड

```
document.title += "!"
```

६ prompt — वापरणाऱ्याला विचारा

`window.prompt(प्रश्न, आधीचे_उत्तर)` एक पेटी दाखवते ज्यात वापरणारा काहीतरी टाईप करतो. तो जे टाईप करतो ते प्रॉम्प्ट परत देते (याला **return value** म्हणतात). म्हणजे कोणी “Hello World” लिहिले तर कोड असा चालतो जणू आपण लिहिले होते `document.title = "Hello World"`.



कोड

```
document.title = window.prompt("Your title?", "");
```

🔧 तुमची पाळी! करून बघा

1. पान उघडताच शीर्षक विचारणारे आणि ते ब्राउझरच्या पट्टीत दाखवणारे पान बनवा.
2. `prompt` च्या दुसऱ्या जागी “नमस्कार” लिहून बघा — काय बदल होतो?

★ लक्षात ठेवा

- प्रॉपर्टीला किंमत द्यायला = वापरतात ("ठेवा").
- += मजकूर जोडते आणि संख्या मिळवते.
- prompt वापरणाऱ्याने लिहिलेले उत्तर परत देते (return value).



धडा ७

कमेंट्स — कोडमधल्या चिठ्या



वहीत कडेला आपण पेन्सिलने “हे महत्वाचे!” असे लिहितो. ते शिक्षक वाचत नाहीत, पण आपल्याला नंतर उपयोगी पडते. कोडमध्येही अशा चिठ्या असतात — त्यांना कमेंट्स म्हणतात.

१ कमेंट्स का महत्वाच्या?

कमेंट्स महत्वाच्या आहेत! कोड मोठा झाला की “ही ओळ काय करते?” हे महिन्याभराने स्वतःलाही आठवत नाही. कमेंट्स आपल्याला आणि इतरांना कोड समजायला मदत करतात. JavaScript कमेंट्स **पूर्णपणे दुर्लक्षित** करते — त्यामुळे कोडच्या वेगावर काहीही परिणाम होत नाही.



बोटू

हा कोड काय करतो ते मला पण आठवत नाही! 😊

काळजी नको, मी लिहून ठेवलंय!



कमेंट

२ एका ओळीची कमेंट

दोन तिरक्या रेषा // लिहिल्या की त्या ओळीचा उरलेला भाग कमेंट बनतो.

// ही कमेंट
आहे



JavaScript दुर्लक्ष करते

कोड

```
// ही एक कमेंट आहे
alert("hello"); // ही सुद्धा कमेंट आहे
```

३ अनेक ओळींची कमेंट

मोठी कमेंट लिहायची असेल तर ती /* आणि */ च्या मध्ये लिहितात. मधला सगळा मजकूर JavaScript वाचत नाही.

कोड

```
/* हा सगळा मजकूर
   JavaScript वाचणार नाही.
   इथे आपण मोठी माहिती लिहू शकतो. */
alert("Hello World");
```

४ कोड तात्पुरता बंद करा (comment out)

कमेंटची आणखी एक मजा: एखादी ओळ **डिलीट न करता** तात्पुरती बंद करता येते. कोडमध्ये चूक सापडत नसेल तर संशयित ओळी एकेक करून कमेंट करून बघा — चूक कोणत्या ओळीत आहे ते कळते!

```
//  
alert("Hello  
World");
```



ओळ झोपली

कोड

```
// जुना संदेश  
// alert("Hello World");  
// नवीन संदेश  
alert("Hello everyone!");
```



तुमची पाळी! करून बघा

1. तुमच्या मागच्या फाइलला `chapter_7.html` म्हणून सेव्ह करा.
2. सुरुवातीला `// This is my first comment` लिहा.
3. ती कमेंट काढून एक मोठी `(/* ... */)` कमेंट लिहा, जी तुमचा कोड काय करतो ते सांगते.

★ लक्षात ठेवा

- `//` एका ओळीची आणि `/* */` अनेक ओळींची कमेंट.
- JavaScript कमेंटकडे बघतही नाही.
- चूक शोधायला कोड “comment out” करून बघा.



धडा ८

जुन्या ब्राउझरपासून स्क्रिप्ट लपवणे



समजा तुम्ही एका मित्राला जादूच्या खेळाची पुस्तिका दिली, पण त्याला जादू माहीतच नाही. तो ते वाचून गोंधळून जाईल! काही ब्राउझरचेही असेच होते.

१ सगळ्यांना JavaScript समजत नाही

खूप जुने ब्राउझर JavaScript समजत नाहीत. काही कंपन्या त्यांच्या कॉम्प्युटरवर JavaScript बंद ठेवतात. अंध लोकांसाठीचे स्क्रीन रीडर आणि Google चे सर्च इंजिन देखील JavaScript चालवत नाहीत. म्हणून आपल्या पानावर JavaScript नसले तरी पान नीट दिसायला हवे.



खूप जुना ब्राउझर



ऑफिसमध्ये JS बंद



स्क्रीन रीडर



Google सर्च रोबोट

२ कोड लपवण्याची युक्ती

JavaScript मध्ये `<!--` ही एका ओळीची कमेंट आहे, पण HTML मध्ये ती अनेक ओळींची कमेंट सुरू करते. याचा फायदा घेऊन आपण कोड असा लपवतो की JavaScript असलेला ब्राउझर तो चालवतो आणि जुना ब्राउझर तो कमेंट समजून सोडून देतो.



जुना ब्राउझर



कमेंट समजून सोडतो



नवा ब्राउझर



कोड चालवतो

कोड

```
<script type="text/javascript">
<!-- hide from older browsers
... तुमचा कोड ...
// stop hiding code -->
</script>
```

३ noscript — दुसरा पर्याय

ज्यांच्या ब्राउझरमध्ये JavaScript चालत नाही, त्यांना काहीतरी उपयोगी दाखवण्यासाठी **<noscript>** टॅग वापरतात. JavaScript चालत असेल तर हा मजकूर दिसत नाही; चालत नसेल तरच दिसतो.



JavaScript चालू

नमस्कार, तुमचे नाव घेऊन स्वागत!

VS



JavaScript बंद



noscript चा मजकूर: "Welcome to our site!"

कोड

```
<noscript>
  <h1>Welcome to our site!</h1>
</noscript>
```

४ चांगल्या सवयी

noscript मध्ये "तुमचा ब्राउझर अपग्रेड करा" असे लिहू नका — काही लोक अंध असतात किंवा त्यांच्या हातात पर्यायच नसतो. त्यापेक्षा तितकेच उपयोगी असे साधे पर्यायी मजकूर द्या. उदाहरणार्थ, JavaScript ने बनवलेल्या मेनूऐवजी साधी HTML यादी.



JS मेनू



JS बंद



साधी HTML यादी



बोटू म्हणतो: आजचे ब्राउझर सगळे JavaScript समजतात, त्यामुळे "लपवण्याची युक्ती" आता क्वचितच लागते. पण **<noscript>** अजूनही उपयोगी आहे.



तुमची पाली! करून बघा

- chapter_8.html मध्ये वरील दोन ओळी (hide / stop hiding) तुमच्या कोडभोवती लावा.
- तुमचा कोड काय करतो ते समजावून सांगणारा एक **<noscript>** भाग लिहा.

★ लक्षात ठेवा

- काही ब्राउझरना JavaScript समजत नाही.
- <!-- ... // --> ने कोड लपवता येतो.
- <noscript> मध्ये चांगला पर्यायी मजकूर द्या.



धडा ९

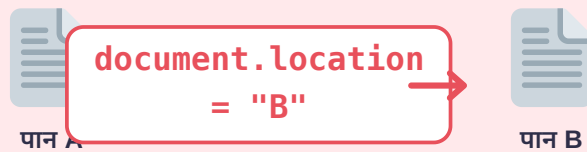
वापरणाऱ्याला आपोआप दुसऱ्या पानावर नेणे



रस्त्यावर “पुढे रस्ता बंद — डावीकडे वळा” अशी पाटी असते आणि सगळे लोक दुसऱ्या रस्त्याने जातात. वेबवरही आपण वापरणाऱ्याला आपोआप दुसऱ्या पानावर पाठवू शकतो. याला **redirect** म्हणतात.

१ document.location — पानाचा पत्ता

पानाचा पत्ता `document.location` या प्रॉपर्टीत असतो. त्यात नवीन पत्ता ठेवला (धडा ६ आठवा!) की ब्राउझर लगेच त्या पानावर जातो.

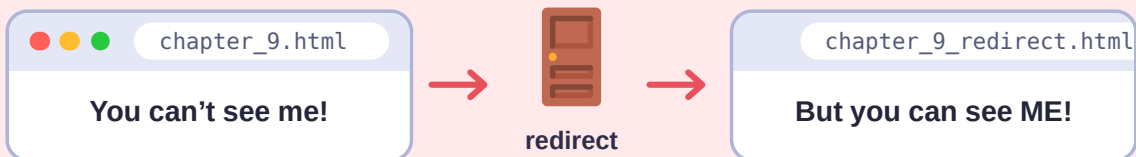


कोड

```
document.location = "http://www.bbc.co.uk/";  
document.location = "chapter_4.html";
```

२ दोन पानांचा प्रयोग

प्रयोगात दोन फाइल बनवतात: `chapter_9.html` (इथे “You can’t see me!” लिहिले आहे आणि **redirect** चा कोड आहे) आणि `chapter_9_redirect.html` (इथे “But you can see ME!” लिहिले आहे). पहिले पान उघडले की तुम्ही आपोआप दुसऱ्या पानावर पोहोचता.



कोड

```
<head>  
  <script type="text/javascript">  
    document.location = "chapter_9_redirect.html";  
  </script>  
</head>
```

३ स्क्रिप्ट कुठे ठेवली यावर बरेच अवलंबून!

script जर पानाच्या **सुरुवातीला (head)** असेल तर पान दिसण्याआधीच redirect होते. पण script जर पानाच्या **शेवटी** ठेवली, तर आधी पानावरचा मजकूर दिसतो आणि नंतर redirect होते. स्क्रिप्टची जागा बदलून प्रयोग करून बघा!

↑ script वर (head मध्ये)



लगेच redirect



मजकूर दिसतच नाही

VS

↓ script खाली (body च्या शेवटी)



आधी मजकूर दिसतो



मग redirect



बोटू म्हणतो: कोड वरून खाली, एकेक ओळ अशा क्रमाने चालतो. म्हणून जागा महत्वाची आहे.



तुमची पाली! करून बघा

1. वर दिलेली दोन पाने बनवा आणि `chapter_9.html` उघडा.
2. redirect चा कोड वेगवेगळ्या जागी ठेवून बघा. "You can't see me!" कधी दिसते?

★ लक्षात ठेवा

- document.location मध्ये नवीन पत्ता ठेवला की पान बदलते.
- याला redirect म्हणतात.
- स्क्रिप्टची जागा काय घडते त्यावर परिणाम करते.



धडा १०

Alert, Prompt आणि Confirm



कॉम्प्युटर तुमच्याशी तीन प्रकारे बोलू शकतो: काहीतरी सांगतो (alert), होय/नाही विचारतो (confirm), आणि उत्तर लिहायला सांगतो (prompt).

१ window — सगळ्यात मोठा ऑब्जेक्ट

ब्राउझरची विंडो म्हणजे **window ऑब्जेक्ट**. पानावरचे बाकी सगळे ऑब्जेक्ट त्याच्या आतच असतात — जसे मोठ्या डब्यात छोटे डबे! म्हणून **window** लिहिणे **ऐच्छिक** आहे.



window



document



write()

कोड

```
window.alert("hi");           // पूर्ण नाव
alert("hi");                   // लहान रूप - दोन्ही सारखेच

window.document.write("hi");
document.write("hi");          // दोन्ही सारखेच
```

२ तीन संवाद-पेट्या

या तिन्ही window च्या मेथड्स आहेत: **alert(संदेश)** फक्त संदेश दाखवते. **confirm(संदेश)** OK / Cancel विचारते. **prompt(संदेश, आधीचे उत्तर)** उत्तर लिहायला जागा देते.

या पानाचा संदेश

Welcome!

OK

या पानाचा संदेश

Do you like JavaScript?

OK

Cancel

या पानाचा संदेश

What is your name?

Aarav

OK

Cancel

३ प्रत्येक पेटी काय परत देते?

प्रत्येक पेटी कोड पुन्हा चालू होताना एक **उत्तर परत देते** (return value).

पेटी	काय परत मिळते?
alert	OK दाबल्यावर true
confirm	OK → true, Cancel → false
prompt	टाइप केलेला मजकूर; काही न लिहिल्यास रिक्त; Cancel → null

४ पेटी बंद होईपर्यंत सगळे थांबते

ह्या पेटीा दिसतात तेव्हा मागचा बाकी सगळा कोड **थांबतो**. तुम्ही OK किंवा Cancel दाबल्यावरच पुढचा कोड चालतो. म्हणून पान दिसण्याआधी “Welcome to my website! Click OK to continue.” दाखवायचे असेल तर alert वापरता येतो.



कोड चालतो



या पानाचा संदेश

Welcome to my website! Click OK to continue.

OK



OK नंतर पान दिसते



तुमची पाळी! करून बघा

- पान उघडताच “Welcome to my website! Click OK to continue.” असा संदेश येईल असे करा.
- आता तिन्ही पेटीा एकेक करून वापरा. `document.write(confirm("OK?"))`; लिहून बघा — काय छापले जाते?

★ लक्षात ठेवा

- alert सांगते, confirm होय/नाही विचारते, prompt उत्तर मागते.
- ह्या window च्या मेथड्स आहेत; window लिहिणे ऐच्छिक.
- confirm true/false आणि prompt टाइप केलेला मजकूर परत देते.



तुमच्या शाळेच्या कपाटात प्रत्येक कप्यावर नाव लिहिलेले असते — “पुस्तके”, “डबा”, “खेळणी”.
JavaScript मध्ये माहिती ठेवायला असेच नाव दिलेले डबे असतात. त्यांना व्हेरिअबल म्हणतात.

१ व्हेरिअबल म्हणजे नाव दिलेला डबा

समजा आपल्याला पानाचे जुने शीर्षक बदलण्याआधी कुठेतरी **साठवून ठेवायचे** आहे, किंवा पान उघडल्याची वेळ अनेक ठिकाणी वापरायची आहे. त्यासाठी आपण स्वतःचे डबे (व्हेरिअबल) बनवतो आणि त्यांना हवे ते नाव देतो.

greeting

"Welcome to my web site"

numPageHits

33

isOpen

true

प्रत्येक डब्याला नाव आणि आत एक किंमत

२ डब्याला नाव कसे द्यायचे?

नावाचे नियम सोपे आहेत: **जागा (space) नको** आणि नाव **अक्षराने** सुरू झाले पाहिजे. आणि लक्षात ठेवा — लहान-मोठी अक्षरे वेगळी मानली जातात: `Message1` आणि `MESSAGE1` हे दोन वेगवेगळे डबे आहेत!

✓ बरोबर नावे



BrowserName



page_name



Message1

VS

✗ चुकीची नावे



my name (जागा आहे)



1stPlace (अंकाने सुरू)

३ डब्यात काय काय ठेवता येते?

व्हेरिअबल तीन प्रकारच्या किमती ठेवू शकतो: **संख्या** (Number), **मजकूर** (String — अवतरण चिन्हात “ ” लिहितात) आणि **खरे/खोटे** (Boolean — फक्त true किंवा false). `1.234` ही संख्या आहे पण `"1.234"` हा मजकूर आहे — दोन्ही वेगळे!

a

12

संख्या

b

"Hello World"

मजकूर

c

true

खरे/खोटे

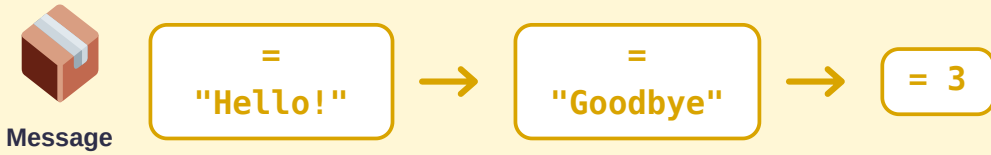
d

"1.234"

मजकूर, संख्या नाही!

४ डबा बनवा आणि किंमत बदला

पहिल्यांदा किंमत ठेवली की डबा आपोआप तयार होतो. नंतर त्यात दुसरी किंमत ठेवता येते — अगदी वेगळ्या प्रकारचीसुद्धा! नवीन डबा बनवताना `let` हा शब्द लिहिणे चांगली सवय आहे.



कोड

```
let numBrowserVersion = 5.5;
let Message = "Hello!";
Message = "Goodbye";    // किंमत बदलली
Message = 3;            // आता संख्या ठेवली
```



बोट्ट म्हणतो: जुन्या पुस्तकांत `let` न लिहिताच थेट `Message = "Hello!";` असे लिहिलेले असते. आजकाल `let` लिहिणे योग्य आहे.

५ व्हेरिअबल वापरताना अवतरण नको!

डब्यातली किंमत वापरायची असेल तर फक्त त्याचे नाव लिहा — **अवतरण चिन्हांशिवाय**. अवतरणात लिहिले तर तो फक्त एक मजकूर होतो.

या पानाचा संदेश

Hello World!

OK

या पानाचा संदेश

Message

OK

डावीकडे `alert(Message)` — उजवीकडे `alert("Message")`

कोड

```
let Message = "Hello World!";
alert(Message);    // डब्यातली किंमत दिसते
alert("Message");  // फक्त "Message" हा शब्द दिसतो
```

६ ऑपरेटर्स — हिशोबाची चिन्हे

व्हेरिअबल्सवर आपण हिशोब करू शकतो. महत्त्वाची गंमत: + चिन्ह संख्यांना बेरीज करते, पण मजकुराला जोडते! म्हणून "1" + "2" चे उत्तर "12" येते, "3" नाही!

चिन्ह	अर्थ
$x + y$	बेरीज (मजकूर असेल तर जोडणे)
$x - y$	वजाबाकी
$x * y$	गुणाकार
x / y	भागाकार
$x \% y$	भागाकार केल्यावर बाकी (remainder)
$-x$	चिन्ह उलटे (+ ↔ -)
$x++$ / $++x$	x मध्ये 1 मिळवा (नंतर / आधी)
$x--$ / $--x$	x मधून 1 वजा करा (नंतर / आधी)

कोड

```
let a = 12;
let b = 13;
let c = a + b;           // c = 25
c += a;                  // c = 37
c = b + " Hello!";      // c = "13 Hello!" (संख्या + मजकूर = मजकूर)
```

🔧 तुमची पाली! करून बघा

- greeting** नावाच्या व्हेरिअबलमध्ये "Welcome to my web site" ठेवा आणि त्याचा alert व शीर्षक (document.write) दोन्हीमध्ये वापर करा — मजकूर एकदाच टाईप करायचा!
- var_1 = "Welcome to"** आणि **var_2 = "my web site"** जोडून **var_3** मध्ये ठेवा. मधली जागा (space) विसरू नका.
- 100 आणि 5.5 वर सगळे ऑपरेटर्स वापरून निकाल **document.write** ने छापवा.

★ लक्षात ठेवा

- व्हेरिअबल = नाव दिलेला डबा.
- तीन प्रकार: संख्या, मजकूर (String), खरे/खोटे (Boolean).
- मजकुरावर + म्हणजे जोडणे: "1" + "2" = "12".



धडा १२

तुलना



तराजूत दोन वस्तू ठेवल्यावर कोणती जड आहे ते कळते. JavaScript मध्येही दोन किमतींची तुलना करता येते — आणि उत्तर नेहमी खरे (true) किंवा खोटे (false) येते!

१ तुलना म्हणजे प्रश्न — उत्तर: खरे/खोटे

तुलना ऑपरेटर दोन किमतींना एकमेकांशी तपासतात. उदाहरणार्थ, “4 हे 10 पेक्षा मोठे आहे का?” — उत्तर: नाही, म्हणजे **false**.



कोड

```
let var_1 = 4;  
let var_2 = 10;  
let var_3 = var_1 > var_2; // var_3 मध्ये false जाते
```

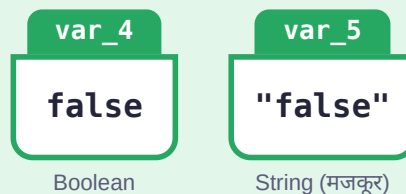
२ तुलनेची चिन्हे

ही चिन्हे नीट लक्षात ठेवा. विशेषतः == (बरोबरी) आणि = (ठेवा) यांत गोंधळ करू नका!

चिन्ह	अर्थ	उदाहरण
x == y	x आणि y सारखे आहेत?	5 == 5 → true
x != y	x आणि y सारखे नाहीत?	5 != 3 → true
x > y	x मोठे आहे?	7 > 2 → true
x >= y	x मोठे किंवा सारखे?	4 >= 4 → true
x < y	x लहान आहे?	3 < 9 → true
x <= y	x लहान किंवा सारखे?	9 <= 2 → false

३ false (खोटे) आणि "false" (मजकूर) वेगळे!

Boolean **false** ही खरी-खोट्याची किंमत आहे. पण **"false"** हा फक्त पाच अक्षरांचा मजकूर आहे. अवतरण चिन्हांमुळे हा फरक पडतो!



४ उलट करायचे? ! वापरा

तुलनेचे उत्तर उलटे करण्यासाठी आधी ! ("नाही") लावतात. "4 मोठे आहे असे नाही" म्हणजे "4 लहान किंवा सारखे आहे". दोन्ही मार्ग बरोबर — तुम्हाला जे वाचायला सोपे वाटते ते वापरा.

$$!(a > b) = a \leq b$$

कोड

```
let a = 4, b = 10;
alert( !(a > b) );    // true
alert( a <= b );     // true – दोन्ही सारखेच
```

तुमची पाली! करून बघा

1. दोन संख्या व्हेरिअबल्समध्ये ठेवा आणि सगळ्या तुलना चिन्हांचे निकाल alert मध्ये दाखवा.
2. आता एक संख्या बदलून मजकूर ठेवा (उदा. "apple") आणि पुन्हा बघा — काय बदल दिसतो?

★ लक्षात ठेवा

- तुलना नेहमी true किंवा false देते.
- == म्हणजे "सारखे?", = म्हणजे "ठेवा".
- ! उत्तर उलटे करतो.



रस्त्यावर सिग्नल बघा: जर दिवा लाल असेल तर थांबा; नाहीतर पुढे जा. आपणही रोज असे निर्णय घेतो. JavaScript ला पण असे निर्णय घ्यायला शिकवूया!

१ निर्णयाचे तीन भाग

आतापर्यंत आपला कोड नेहमी तसाच चालत होता. निर्णय घेण्यासाठी तीन गोष्टी लागतात: ① **अट (test)** — खरी की खोटी? ② अट खरी असेल तर करायचे काम. ③ अट खोटी असेल तर करायचे काम (हे ऐच्छिक).

✓ अट खरी (true)



हिरवा → जा!

VS

✗ अट खोटी (false)



else → थांबा!

कोड

```
if ( अट )
{
  // अट खरी असेल तर हे चालेल
}
else
{
  // अट खोटी असेल तर हे चालेल
}
```

२ पहिले उदाहरण

दोन संख्यांपैकी मोठी कोणती ते सांगूया.



var_1 >
var_2 ?



या पानाचा संदेश

Variable 1 is greater

OK

नाहीतर

या पानाचा संदेश

Variable 2 is greater

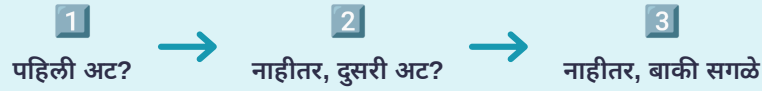
OK

कोड

```
if ( var_1 > var_2 )
{
  alert("Variable 1 is greater");
}
else
{
  alert("Variable 2 is greater");
}
```

३ else if — आणखी पर्याय

वरच्या कोडमध्ये दोन संख्या **सारख्या** असतील तर चुकीचा संदेश येतो! म्हणून मध्ये आणखी एक अट घालतात: **else if**. असे हवे तितके else if ठेवू शकता. सगळ्यात शेवटचा else “बाकी सगळ्या बाबतीत” असतो.



कोड

```
if ( var_1 > var_2 )
{
  alert("Variable 1 is greater");
}
else if ( var_1 < var_2 )
{
  alert("Variable 2 is greater");
}
else
{
  alert("The variables are equal");
}
```

४ लहान प्रोग्राम — ternary ऑपरेटर

एकच किंमत ठरवायची असेल तर लांब if-else ऐवजी एक छोटा मार्ग आहे — **? :**. “अट ? खरी असेल तर ही किंमत : नाहीतर ही”. याला **ternary operator** म्हणतात.

```
(var_1 > 4) ? var_1 : 4
```

कोड

```
let var_2 = (var_1 > 4) ? var_1 : 4;
// var_1 आणि 4 यांपैकी जे मोठे ते var_2 मध्ये ठेवा
```



बोटू म्हणतो: जर if नंतर फक्त एकच ओळ असेल तर { } वगळता येतात. पण नंतर ओळी वाढवल्या तर घोटाळा होतो — म्हणून नेहमी { } वापरणे चांगले.

५ परीक्षेचा निकाल — मोठा प्रयोग

`exam_result` मध्ये 0 ते 100 मधली संख्या ठेवा आणि खालीलप्रमाणे संदेश छपा.

गुण	संदेश
90 किंवा जास्त	Excellent. Pass with Distinction.
70 ते 89	Well Done. Pass with Honours
55 ते 69	Just passed.
54 किंवा कमी	Failed. Do better next time.

कोड

```
let exam_result = 82;
if ( exam_result >= 90 ) { document.write("Excellent. Pass with Distinction."); }
else if ( exam_result >= 70 ) { document.write("Well Done. Pass with Honours"); }
else if ( exam_result >= 55 ) { document.write("Just passed."); }
else { document.write("Failed. Do better next time."); }
```



तुमची पाळी! करून बघा

- दोन संख्यांपैकी मोठी कोणती ते `alert` मध्ये सांगणारा कोड लिहा.
- `prompt` ने नाव विचारा. तुमचे नाव असेल तर “नमस्कार, मालक!”, मित्राचे नाव असेल तर वेगळा संदेश आणि बाकी सगळ्यांना “नमस्कार पाहुणे!” दाखवा.
- तुम्ही `greeting` नावाच्या व्हेरिअबलमध्ये संदेश ठेवून `document.write` फक्त एकदाच वापरू शकता का?

★ लक्षात ठेवा

- `if (अट) { ... } else { ... }` — निर्णय घ्यायला.
- `else if` ने पर्याय वाढवता येतात.
- (अट) ? खरे : खोटे — लहान रूप.



धडा १४

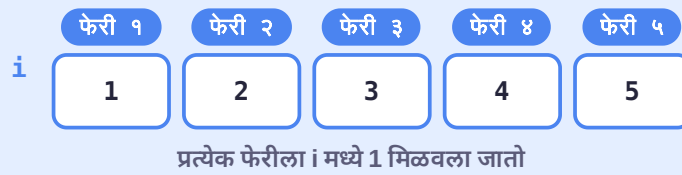
लूप्स — पुन्हा पुन्हा करा



शिक्षकांनी तुम्हाला “मी वेळेवर येईन” हे १०० वेळा लिहायला सांगितले तर? 🤖 कॉम्प्युटरला सांगा: “हे १०० वेळा कर!” — काम संपले! याला लूप म्हणतात.

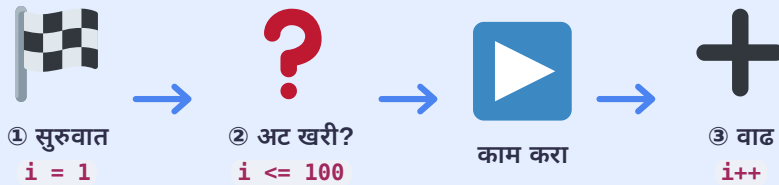
१ मोजणारा डबा — i

लूपमध्ये आपण “किती वेळा झाले” हे मोजायला एक डबा वापरतो. प्रोग्रामर त्याला सहसा i, j किंवा k नाव देतात. प्रत्येक फेरीला त्यात 1 मिळवतात: $i = i + 1$. याचे छोटे रूप: $i++$ (आणि वजा करायला $i--$).



२ for लूप — तीन भाग

for लूपच्या कंसात तीन गोष्टी असतात: ① **सुरुवात** — i कुठून सुरू होईल. ② **अट** — लूप चालू ठेवायचा का? (खरी असेल तोपर्यंत चालतो) ③ **वाढ** — प्रत्येक फेरीनंतर काय करायचे.



कोड

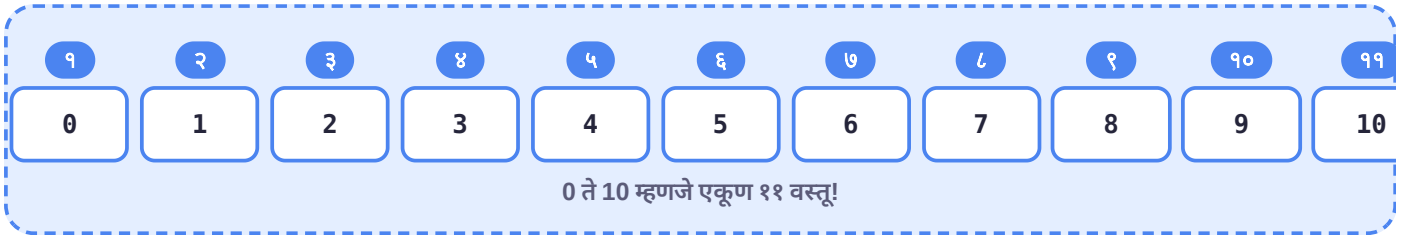
```
for ( let i = 1; i <= 100; i++ )
{
  document.write("<p>" + i + "</p>");
}
```



निकाल: पानावर 1 ते 100 पर्यंत संख्या एकेका ओळीत छापल्या जातात.

३ मोजणी ० पासून!

प्रोग्रामिंगमध्ये मोजणी बहुतेकदा **0 पासून** सुरू करतात. त्यामुळे गोंधळ होऊ शकतो: 0 पासून 10 पर्यंत मोजले तर प्रत्यक्षात **11** वस्तू मोजल्या जातात! 0 पासून सुरू केले तर अट `i < 300` (300 पेक्षा लहान) असते, आणि 1 पासून सुरू केले तर `i <= 300`.



४ सावधान — न संपणारा लूप

जर अट **कधीच खोटी होत नसेल** तर लूप कधीच संपत नाही आणि ब्राउझर अडकतो! उदाहरणार्थ `5 > 4` नेहमी खरे असते.



`for ( i = 0; 5 > 4; i++ )`

कधीच थांबत नाही!

कोड

```
// असे कधी लिहू नका:  
for ( let i = 0; 5 > 4; i++ )  
{  
  // अट नेहमी true — लूप संपतच नाही!  
}
```

५ पाढे बनवूया — गुणाकार तक्ता

लूपने आपण कोणताही पाढा सहज छापू शकतो. HTML टेबल वापरले की = आणि × चिन्हे सरळ रेषेत दिसतात.

अंक	चिन्ह	गुणक	=	उत्तर
7	×	1	=	7
7	×	2	=	14
7	×	3	=	21
...				...

कोड

```
document.write("<table>");  
for ( let i = 1; i <= 10; i++ )  
{  
  document.write("<tr><td>7</td><td>×</td><td>" + i +  
    "</td><td>=</td><td>" + (7 * i) + "</td></tr>");  
}  
document.write("</table>");
```



तुमची पाली! करून बघा

- 1 ते 20 पर्यंत संख्या छापणारा लूप लिहा.
- prompt ने एक संख्या विचारून तिचा पाढा छपा. पान पानाच्या मध्यभागी दिसण्यासाठी `<center>` किंवा CSS वापरून बघा.

★ लक्षात ठेवा

- for (सुरुवात; अट; वाढ) { ... }
- मोजणी बहुतेकदा 0 पासून सुरू होते.
- अट कधीच खोटी झाली नाही तर लूप संपत नाही.



धडा १५

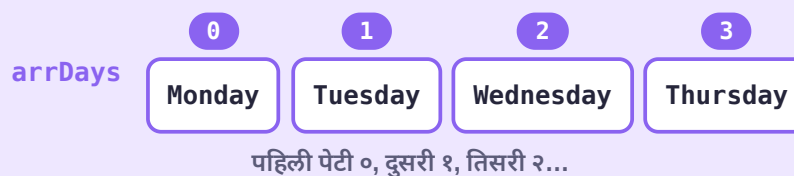
१५ अरे — डब्यांची रांग



ऑफिसमध्ये भिंतीवर टपाल-पेट्यांची एक रांग असते. प्रत्येक पेटीला क्रमांक असतो. एकच पेटी म्हणजे व्हेरिअबल, आणि पेट्यांची रांग म्हणजे अरे!

१ अरे म्हणजे काय?

एका व्हेरिअबलमध्ये फक्त **एकच** किंमत राहते. पण आठवड्याच्या सोमवारची सगळी कामे एकत्र ठेवायची असतील तर? त्यासाठी अरे वापरतात. अरे एक खास ऑब्जेक्ट आहे ज्यात **अनेक किमती** ठेवता येतात. प्रत्येक पेटीचा क्रमांक असतो — त्याला **index** म्हणतात आणि तो **० पासून** सुरू होतो.



२ अरे कसा बनवायचा?

अरे बनवण्याचे अनेक मार्ग आहेत. सगळ्यात जास्त वापरला जाणारा मार्ग म्हणजे चौकोनी कंस [].



कोड

```
let arrDays = new Array(); // रिकामा अरे
let arrDays2 = new Array("Monday", "Tuesday");
let arrDays3 = ["Monday", "Tuesday"]; // सगळ्यात सोपा मार्ग
```

३ पेटी उघडा — किंमत वाचा आणि बदला

अरेचे नाव लिहून चौकोनी कंसात क्रमांक लिहिला की ती पेटी उघडते. **अवतरण चिन्हे नकोत** क्रमांकाभोवती! तसेच त्याच पद्धतीने किंमत ठेवताही येते. शेवटी नवीन किंमत जोडायला **push()** मेथड वापरतात.

कोड

```
alert(arrDays[0]); // "Monday" दिसते
arrDays[2] = "Wednesday"; // तिसऱ्या पेटीत किंमत ठेवा
arrDays.push("Thursday"); // रांगेच्या शेवटी नवीन पेटी
```

४ length आणि sort

प्रत्येक अरेला काही खास गोष्टी असतात. **length** सांगते की रांगेत किती पेट्या आहेत. **sort()** सगळ्या किमती अक्षरांच्या क्रमाने लावते (मूळ अरेतच बदल होतो!).



कोड

```
let how_many_days = arrDays.length; // रांगेत किती पेट्या?  
arrDays.sort(); // अक्षरक्रमाने लावा
```

५ सगळ्यात महत्वाची युक्ती: अरे + लूप

पेट्यांच्या रांगेतून एकेक पेटी उघडून बघायला **for लूप** वापरतात. `arr[i]` म्हणजे “i व्या पेटीतली किंमत”. आणि `arr.length` वापरल्याने आपला लूप कोणत्याही आकाराच्या अरेसाठी चालतो!



कोड

```
let months = ["March", "January", "May", "February", "April"];  
months.sort();  
for ( let i = 0; i < months.length; i++ )  
{  
  document.write(months[i] + "<br>");  
}
```

📺 **निकाल:** April, February, January, March, May — अक्षरक्रमाने एकेक ओळीत.



बोटू म्हणतो: `months.length` वापरल्याने, तुम्ही अरे बदलला तरी लूप बदलावा लागत नाही!



तुमची पाली! करून बघा

- वर्षाचे १२ महिने अरेत ठेवा, sort करा आणि लूपने छपा.
- आता तोच कोड आठवड्याच्या वारांच्या अरेसाठी चालवा. लूपचा कोड बदलावा लागला का?

★ लक्षात ठेवा

- अरे = अनेक किमतींची रांग; index ० पासून.
- `arr[i]` ने `i` वी किंमत मिळते, `arr.length` ने एकूण संख्या.
- अरे + for लूप = सगळ्या किमतींवर काम.



धडा १६

१६

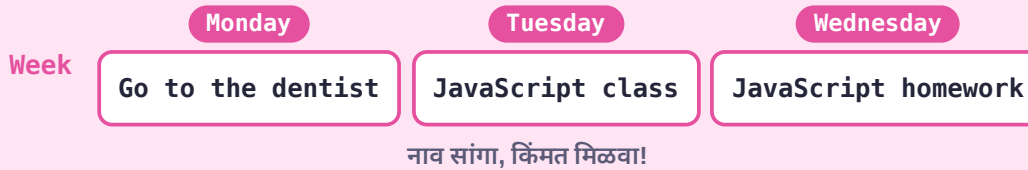
नावाचे डबे — Associative Array



ऑफिसच्या पेठ्यांना फक्त ०, १, २ क्रमांक न देता “सोमवार”, “मंगळवार” अशी नावे दिली तर? पेटी शोधणे किती सोपे होईल!

१ क्रमांकाऐवजी नाव

आतापर्यंत अरेच्या पेठ्या क्रमांकाने उघडल्या. पण त्यांना **नावे** देऊनही उघडता येते. याला **associative array** म्हणतात. येथे नाव म्हणजे “की” (key) आणि आतली किंमत म्हणजे “value”.



कोड

```
let Week = {};  
Week["Monday"] = "Go to the dentist";  
Week["Tuesday"] = "Attend JavaScript class";  
Week["Wednesday"] = "JavaScript homework";  
  
alert(Week["Wednesday"]); // "JavaScript homework"
```



बोटू म्हणतो: जुन्या पुस्तकात असे नावाचे डबे `new Array()` ने बनवले आहेत. आजकाल असे डबे `{ }` ने बनवतात. कल्पना सारखीच आहे!

२ ऑब्जेक्टसारखे वापरा (डॉट)

नावांचे डबे म्हणजे प्रत्यक्षात ऑब्जेक्टच! त्यामुळे डॉट (.) वापरूनही किंमत वाचता येते — जणू ती प्रॉपर्टी आहे.

Week["Wednesday"] = Week.Wednesday

कोड

```
alert( Week.Wednesday ); // वरच्यासारखेच उत्तर
```

३ नाव आणि क्रमांक — गोंधळ करू नका

क्रमांकाच्या अंरंमध्ये वारांची नावे **किंमती** होती. नावांच्या डब्यात वारांची नावे **की** आहेत. हे वेगळे आहे! जर तुम्ही क्रमांकाच्या अंरंत नावाने किंमत ठेवली, तर अंर ३ पेट्यांचा होईल, २ चा नाही.

क्रमांकाचा अंर

0 → "Monday"

1 → "Tuesday"

VS

नावाचे डबे

"Monday" → "Go to the dentist"

"Tuesday" → "JavaScript class"

४ सगळ्या नावांवर लूप — for...in

नावांचे डबे असतील तर सगळ्या नावांवरून फिरण्यासाठी **for...in** लूप वापरतात. **day** मध्ये एकेक नाव येते, आणि **Week[day]** ने त्याची किंमत मिळते.



for...in



प्रत्येक वार



वार: काम

कोड

```
for ( let day in Week )
{
  document.write(day + ": " + Week[day] + "<br>");
}
```

निकाल: Monday: Go to the dentist • Tuesday: Attend JavaScript class ...

तुमची पाली! करून बघा

- सात वारांसाठी सात कामे असलेला Week बनवा आणि for...in ने कॅलेंडर छपा.
- prompt ने वार विचारा आणि त्या दिवसाचे काम दाखवा.

★ लक्षात ठेवा

- नावाचे डबे = नावाने किंमत मिळवणे.
- Week["Monday"] किंवा Week.Monday — दोन्ही चालते.
- for...in ने सगळ्या नावांवरून फिरता येते.



धडा १७

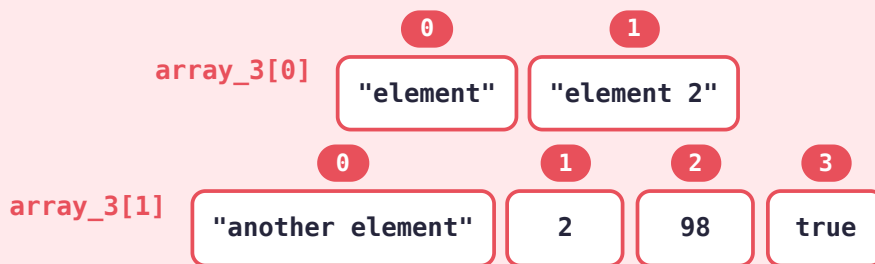
दोन मिती असलेले अरे



एका इमारतीत अनेक मजले आहेत आणि प्रत्येक मजल्यावर खोल्यांची रांग आहे. खोली शोधायला दोन गोष्टी सांगाव्या लागतात: कोणता मजला आणि कोणती खोली. हेच दोन मितीचे अरे!

१ अरेच्या आत अरे

2D अरे म्हणजे असा अरे ज्याची प्रत्येक पेटी स्वतःच एक अरे आहे. म्हणजे टपाल-पेट्यांच्या अनेक रांगा एकावर एक ठेवलेल्या. प्रत्येक रांग वेगळ्या लांबीची असू शकते.



२ दोन कंस — दोन क्रमांक

किंमत मिळवायला दोन चौकोनी कंस लिहितात. पहिला कंस सांगतो कोणती रांग, दुसरा कंस सांगतो त्या रांगेतली कोणती पेटी.

`array_3[1][2]` म्हणजे रांग क्रमांक १ (दुसरी रांग, `array_2`) आणि त्यातली पेटी क्रमांक २ (तिसरी पेटी) = 98.



कोड

```
let array_1 = ["element", "element 2"];
let array_2 = ["another element", 2, 98, true];
let array_3 = [array_1, array_2];

alert( array_3[1][2] );    // 98 दिसते
```



बोट्टू म्हणतो: क्रमांक नेहमी ० पासून सुरू होतात! म्हणून `[१][२]` म्हणजे “दुसरी रांग, तिसरी पेटी”.

३ नावांचे डबे सुद्धा चालतात

रंगांना क्रमांकाऐवजी नावे देता येतात आणि डॉट वापरूनही किंमत वाचता येते.

कोड

```
let array_3 = {};  
array_3["firstArray"] = array_1;  
array_3["secondArray"] = array_2;  
  
alert( array_3["secondArray"][2] ); // 98  
alert( array_3.secondArray[2] ); // 98 – तेच उत्तर
```

४ आणखी खोल — ३ मिती!

अॅरेच्या आत अॅरेच्या आत अॅरे असे हवे तितके खोलही जाता येते. उदाहरण: **महिना** → **आठवडा** → **दिवस** → **तास**. यासाठी तीन-चार कंस लागतात.



महिना



आठवडा



दिवस



तास (8am–5pm)



तुमची पाली! करून बघा

- धडा १६ मधला Week अॅरे घेऊन असे ४ आठवडे बनवा आणि ते एका Month अॅरेत ठेवा.
- prompt ने आठवडा, वार आणि तास विचारा आणि ते काम दाखवा. काम सापडले नाही तर चूक दाखवणारा संदेश द्या.

★ लक्षात ठेवा

- 2D अॅरे = अॅरेचा अॅरे.
- array[रंग][पेटी] — दोन कंस, दोन क्रमांक.
- मिती हव्या तेवढ्या वाढवता येतात.



तुम्ही पुस्तकात एखादा शब्द शोधता तेव्हा काय करता? पाने उलटता आणि शब्द शोधता. JavaScript पण मजकुराच्या आत शब्द शोधू शकते!

१ फक्त “सारखे” पुरेसे नाही

समजा आपण वापरणाऱ्याचे नाव “Karen” शी जुळवतो. ती “Karen Aaronofsky” असे पूर्ण नाव लिहिते — तर == ने आपण तिला ओळखणार नाही. आपल्याला मजकुराच्या आत “Karen” हा शब्द आहे का ते बघायचे आहे. यालाच **substring शोध** म्हणतात.



“Karen”

+



“Karen Aaronofsky”



सापडला!

२ indexOf — शब्द कुठे आहे?

प्रत्येक मजकुरावर `indexOf(“शब्द”)` वापरता येते. ती सांगते की शब्द कितव्या अक्षरापासून सुरू होतो (० पासून मोजून!). सापडला नाही तर उत्तर -1.

Karen
onofsky"

0	1	2	3	4	5	6	7	8	9
K	a	r	e	n		A	a	r	o

`indexOf("Karen")` → 0 • `indexOf("Aaronofsky")` → 6 • `indexOf("Anisa")` → -1

कोड

```
let var_1 = "Karen Aaronofsky";
alert( var_1.indexOf("Karen") );           // 0
alert( var_1.indexOf("Aaronofsky") );      // 6
alert( var_1.indexOf("Anisa") );           // -1 (सापडला नाही)
```

३ लहान-मोठी अक्षरे — एकच करा

JavaScript साठी "Karen" आणि "karen" वेगळे आहेत. म्हणून तुलना करण्याआधी दोन्ही मजकूर लहान अक्षरांत (`toLowerCase()`) बदलतात. मग फरक पडत नाही!

"KAREN" .toLowerCase() → "karen"

कोड

```
let real_name = "karen";
let name = prompt("Your name?", "");
if ( name.toLowerCase().indexOf(real_name) > -1 )
{
    alert("Hello Karen!");
}
else
{
    alert("Welcome " + name);
}
```

४ मजकुराच्या उपयुक्त मेथड्स

मजकुरावर वापरता येणाऱ्या काही मेथड्स:

मेथड	काय करते?
<code>indexOf("str")</code>	शब्द कितव्या जागेवर सापडतो ते सांगते
<code>charAt(x)</code>	x व्या जागेवरचे अक्षर देते
<code>toLowerCase()</code>	सगळी अक्षरे लहान करते
<code>toUpperCase()</code>	सगळी अक्षरे मोठी करते
<code>match(/exp/)</code>	नमुन्याशी जुळते का ते (regular expression)

५ अक्षरांचा नमुना — Regular Expression

ईमेल पत्ता बघितला की आपल्याला लगेच कळते की तो ईमेल आहे — कारण त्याचा एक नमुना (pattern) असतो: **[काही अक्षरे] @ [काही अक्षरे] . [काही अक्षरे]**. हा नमुना JavaScript ला सांगण्याच्या पद्धतीला **regular expression** म्हणतात आणि ती `/` च्या मध्ये लिहितात.

काही अक्षरे

@

काही अक्षरे

.

काही अक्षरे



ईमेल!

कोड

```
/^.+@.+\\.+$/  
// ^ = सुरुवात      .+ = कोणतीही एक किंवा जास्त अक्षरे  
// @ = हे अक्षर      \. = खरा टिंब (.)  
// $ = शेवट
```



बोट्टू म्हणतो: Regular expression कठीण वाटते — काळजी करू नका! फक्त कल्पना समजा: “नमुना सांगून मजकूर तपासता येतो.”

६ ईमेल तपासणारा कोड

वापरणाऱ्याने लिहिलेला मजकूर ईमेलसारखा दिसतो का ते तपासूया.

या पानाचा संदेश

Your email?

aarav@example.com

OK

Cancel

या पानाचा संदेश

valid email address

OK

कोड

```
let var_1 = prompt("Your email?", "");  
if ( var_1.match(/^.+@.+\\.+$/) )  
{  
    alert("valid email address");  
}  
else  
{  
    alert("are you sure?");  
}
```



बोट्टू म्हणतो: ही तपासणी अजून परिपूर्ण नाही — “-@-.” सुद्धा पास होतो! पुढे आपण ती आणखी चांगली करू.



तुमची पाळी! करून बघा

1. prompt ने एक मजकूर विचारा आणि त्यावर सगळ्या मेथड्स वापरून निकाल छाप.
2. तो ईमेलसारखा दिसतो का ते तपासा: "That's an email address" किंवा "That doesn't look like an email address to me!" आणि चुकीचा मजकूर पुन्हा दाखवा.

★ लक्षात ठेवा

- indexOf शब्द शोधते; सापडला नाही तर -1.
- तुलनेआधी toLowerCase() ने अक्षरे एकसारखी करा.
- regular expression म्हणजे अक्षरांचा नमुना.



धडा १९

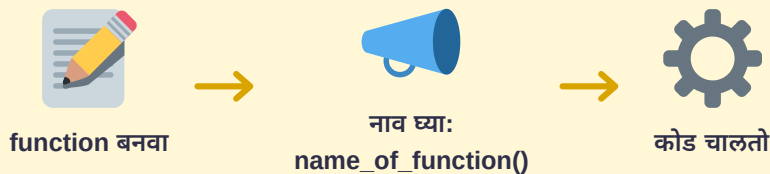
फंक्शन्स वापरूया



ज्यूस-मशीनमध्ये तुम्ही फळे टाकता, बटण दाबता आणि ज्यूस बाहेर येतो. फंक्शन अगदी असेच एक मशीन आहे — आत माहिती टाका, काम होते आणि उत्तर बाहेर येते!

१ फंक्शन म्हणजे नाव दिलेले काम

फंक्शन म्हणजे JavaScript च्या काही स्टेटमेंट्सचा गट, ज्याला आपण नाव दिले आहे. नाव व्हेरिअबलसारखेच ठेवतात. फंक्शनमधला कोड आपोआप चालत नाही — आपण त्याला “बोलावले” (call केले) तरच चालतो. बोलावण्यासाठी नावानंतर कंस लिहितात.



कोड

```
function name_of_function( )
{
  // तुमचा कोड येथे
}

name_of_function();    // फंक्शन बोलावले
```

२ फंक्शनला माहिती द्या — आर्ग्युमेंट

कंसात माहिती दिली की फंक्शन वेगळ्या वेगळ्या माहितीसोबत काम करू शकते. ज्यूस-मशीनला वेगळी फळे दिली की वेगळा ज्यूस! आपण दिलेल्या माहितीला **आर्ग्युमेंट** म्हणतात.



“Anisa”



greet_user()



या पानाचा संदेश

Hello Anisa

OK

कोड

```
function greet_user( username )
{
  let message = "Hello " + username;
  alert(message);
}

greet_user("Anisa");
greet_user( prompt("Name?", "") );
```

३ फंक्शन उत्तर परत देते — return

कधीकधी फंक्शनने आपल्याला उत्तर **परत द्यावे** असे वाटते. त्यासाठी **return** लिहितात. उदाहरण: कोणी दिलेला मजकूर ईमेलसारखा दिसतो का? उत्तर true किंवा false.



कोड

```
function check_email( address )
{
  return address.match(/^.+@.+\.+$/);
}

if ( check_email(address) )
{
  // ईमेलचे काम करा
}
```

४ फंक्शन का वापरायचे?

समजा तुम्हाला ईमेल तपासण्याची आणखी चांगली पद्धत सापडली. फंक्शन नसेल तर पूर्ण कोडमध्ये प्रत्येक ठिकाणी शोधून बदलावे लागेल. फंक्शन असेल तर **एकाच जागी** बदल केला की सगळीकडे आपोआप सुधारणा होते! तसेच कोड वाचायला सोपा होतो.

 **✗ फंक्शनशिवाय**
१० ठिकाणी बदलावे लागते

VS

 **✓ फंक्शन वापरून**
एकाच जागी बदल
 सगळीकडे आपोआप



बोटू म्हणतो: फंक्शन आधी लिहावे (सहसा <head> मध्ये) आणि नंतर बोलवावे — जेणेकरून ते बोलावण्याआधी तयार असेल.



तुमची पाळी! करून बघा

1. **head** मध्ये `show_message(message)` फंक्शन बनवा जे alert दाखवते आणि **body** मध्ये वेगवेगळ्या संदेशांसह त्याला बोलवा.
2. `get_message()` फंक्शन बनवा जे prompt ने संदेश विचारून तो **return** करते: `message = get_message();`
3. `show_message` बदलून alert ऐवजी `document.write` करा — **body** चा कोड न बदलता!

★ लक्षात ठेवा

- `function नाव() { ... }` — बनवा आणि `नाव()` — बोलवा.
- कंसात आर्ग्युमेंट देतात; `return` ने उत्तर परत येते.
- फंक्शनमुळे कोड एकाच जागी सुधारता येतो.



धडा २०

लॉजिकल ऑपरेटर्स — आणि / किंवा



खजिन्याची पेटी उघडायला तुम्हाला दोन्ही चाव्या लागतात की कोणतीही एक चावी चालते? दोन्ही ठिकाणी उत्तर वेगळे! JavaScript मध्ये यासाठी लॉजिकल ऑपरेटर्स असतात.

१ एकापेक्षा जास्त अटी

आतापर्यंत आपण एका वेळी एकच अट तपासली. पण “x हे दोन संख्यांच्या मध्ये आहे का?” किंवा “दर मंगळवारी रात्री ८ वाजता काम कर” अशा वेळी **दोन अटी** एकत्र तपासाव्या लागतात — मंगळवार आहे का **आणि** रात्रीचे ८ वाजले आहेत का? यासाठी **&& (आणि)** आणि **|| (किंवा)** वापरतात.



आज मंगळवार?



रात्रीचे ८?



दोन्ही खरे तरच!

२ && — आणि (दोन्ही खरे हवेत)

&& फक्त तेव्हाच true देतो जेव्हा **दोन्ही** अटी खऱ्या असतात. पहिलीच अट खोटी असेल तर JavaScript दुसरी बघतही नाही (कारण उत्तर आधीच ठरले!).



चावी १

+



चावी २



पेटी उघडते

कोड

```
( 100 > 10 && 5 < 8 )    // true  (दोन्ही खरे)
( 100 > 200 && 4 < 9 )    // false (पहिले खोटे)
```

३ || — किंवा (एक खरी तरी पुरे)

|| तेव्हा true देतो जेव्हा कोणतीही एक अट खरी असते. दोन्ही खोट्या असतील तरच false. उदाहरण: “महिन्याचा पहिला मंगळवार किंवा ३ तारीख” — जर दोन्ही एकाच दिवशी आले तरी चालते.



कोड

```
( 100 > 10 || 9 < 8 )    // true  (पहिले खरे)
( 100 > 200 || 4 > 9 )   // false (दोन्ही खोटे)
```

४ सत्य-तक्ता (Truth Table)

दोन अटी A आणि B असतील तेव्हा निकाल असा येतो:

A	B	A && B (आणि)	A B (किंवा)
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

५ प्रयोग: ब्राउझरमध्ये कोणते प्लगइन आहे?

मूळ प्रयोगात एका मोठ्या 2D अ‍ॅरेंट ब्राउझरचे प्लगइन (Flash, QuickTime इ.) दिलेले असतात. आपण लूप वापरून तपासतो की Flash आहे का, QuickTime आहे का, आणि **दोन्ही** आहेत का. येथे एक छोटे उदाहरण देत आहोत.



कोड

```
let available_plugins = [ ["Flash", "10"], ["QuickTime", "7"], ["PDF", "9"] ];

function plugin_exists( name )
{
  for ( let i = 0; i < available_plugins.length; i++ )
  {
    if ( available_plugins[i][0] == name ) { return true; }
  }
  return false;
}

if ( plugin_exists("Flash") && plugin_exists("QuickTime") )
  alert("Both QuickTime and Flash");
else
  alert("One of QuickTime or Flash is missing");
```



बोट्ट म्हणतो: Flash आता वापरले जात नाही, पण “अ‍ॅरेंट शोधा आणि && ने दोन अटी जोडा” ही कल्पना कायम उपयोगी आहे!



तुमची पाळी! करून बघा

1. `flash_exists()` आणि `quicktime_exists()` अशी दोन फंक्शन्स लिहा आणि तिसऱ्या फंक्शनमध्ये दोन्ही && ने वापरा.
2. तुमचे वय १० ते १२ च्या मध्ये आहे का हे && वापरून तपासा.

★ लक्षात ठेवा

- && = आणि (दोन्ही खरे हवेत).
- || = किंवा (एक खरे पुरे).
- अटी जोडून मोठे निर्णय घेता येतात.



दारावरची बेल कोणी दाबली की “टिंग-टॉंग!” आवाज येतो. बेल दाबणे = इव्हेंट (घटना) आणि आवाज येणे = इव्हेंट हँडलर (उत्तर). JavaScript मध्येही पानावर काही घडले की आपण कोड चालवू शकतो!

१ इव्हेंट म्हणजे काय?

आतापर्यंत आपला कोड पान उघडताच चालत होता. पण खरी मजा तेव्हा येते जेव्हा कोड वापरणाऱ्याच्या **कृतीला** उत्तर देतो. वापरणारा जे काही करतो — क्लिक, माउस हलवणे, फॉर्म पाठवणे, पान सोडणे — तेव्हा ब्राउझर JavaScript ला सांगतो: “एक **इव्हेंट** घडला!”



क्लिक



माउस वर नेला



फॉर्म पाठवला



पान उघडले



पान सोडले

२ इव्हेंट हँडलर्सची यादी

इव्हेंट हँडलर म्हणजे एक कोडचा तुकडा जो ठरावीक इव्हेंट घडताच चालतो. हँडलर्सची नावे “on” ने सुरू होतात.

इव्हेंट हँडलर	केव्हा चालतो?
onload	पान (किंवा चित्र) लोड झाल्यावर
onunload	वापरणारा पान सोडून जाताना
onmouseover	माउस त्या जागेवर आल्यावर
onmouseout	माउस त्या जागेतून बाहेर गेल्यावर
onclick	डावे बटण क्लिक केल्यावर
onmousedown	माउसचे बटण खाली दाबल्यावर
onmouseup	दाबलेले बटण सोडल्यावर

३ onclick = खाली दाबणे + वर सोडणे

onclick तेव्हाच होते जेव्हा माउसचे बटण **त्याच जागी** दाबले (mousedown) आणि सोडले (mouseup). जर तुम्ही बटण दाबले आणि माउस दुसरीकडे नेऊन सोडले, तर क्लिक झाले असे धरले जात नाही.



mousedown

+



mouseup (त्याच जागी)

=



click!

४ हँडलर टॅगला जोडा

हँडलर आपण HTML टॅगमध्ये (विशेषतः `<img>`, `<a>` आणि `<body>`) असे लिहितो: `onclick="काम"`. आपण फंक्शन बोलावू शकतो किंवा थेट JavaScript लिहू शकतो. **अवतरण चिन्हांचा नियम:** बाहेर “ ” वापरले तर आत ‘ ’ वापरा आणि उलट.



लिंक

onclick



फंक्शन / कोड

कोड

```
<a href="#" onclick="a_function();">link</a>

<a href="#" onclick="alert('hello');">link</a>
```

५ पान बदलू नका — आधी विचारा! (return false)

लिंक्वर क्लिक केले की ब्राउझर साधारणपणे त्या पानावर जातो. हँडलर आधी चालतो आणि मग ते “नेहमीचे काम” होते. जर आपल्याला ते **रोखायचे** असेल तर हँडलर `false` परत करतो. हँडलरच्या आधी `return` लिहायला विसरू नका!



क्लिक



या पानाचा संदेश

This will take you away from our site. Are you sure?

OK

Cancel



OK → जाते
Cancel → थांबते

कोड

```
function confirm_link( )
{
  let check = confirm("This will take you away from our site. Are you sure?");
  return check;    // true किंवा false
}

<a href="http://www.google.com/" onclick="return confirm_link();">Google</a>
```

६ माउसचा जादू-खेळ

पानावर लिंक ठेवा आणि माउस तिच्यावर नेल्यावर पानाचे शीर्षक बदला. माउस दूर नेल्यावर पुन्हा आधीचे शीर्षक!

कोड

```
function set_title( msg )
{
  document.title = msg;
}

<a href="#" onmouseover="set_title('Hello!');"
  onmouseout="set_title('Chapter 21');">माउस येथे आणा</a>
```



बोट्टू म्हणतो: जुन्या पुस्तकात ही जादू ब्राउझरच्या खालच्या पट्टीसाठी (`window.status`) केली आहे. आजचे ब्राउझर ती पट्टी बदलू देत नाहीत, म्हणून आपण शीर्षक बदलले.



तुमची पाळी! करून बघा

१. वर दिलेली `set_title` लिंक बनवा आणि माउस वर नेऊन/काढून बघा.
२. लिंक एखाद्या खऱ्या वेबसाइटला जोडा आणि `confirm_link()` वापरून क्लिक रोखून दाखवा.

★ लक्षात ठेवा

- इव्हेंट = घडलेली घटना; हँडलर = त्याला उत्तर देणारा कोड.
- onclick, onmouseover, onmouseout, onload...
- हँडलरने false परत केले (return) तर नेहमीचे काम थांबते.



धडा २२

चित्रांशी खेळ



वर्गात प्रत्येक मुलाला नाव असते — “राजू”, “सीमा”. शिक्षकांना सीमाला बोलवायचे असेल तर ते तिचे नाव घेतात. पानावरच्या चित्राला पण नाव (id) देता येते, मग JavaScript ला सांगता येते — “त्या चित्राला बोलव!”

१ चित्राला id (नाव) द्या

HTML मध्ये कोणत्याही गोष्टीला id ने नाव देता येते. चित्राला असे नाव दिले की JavaScript ला ते लगेच सापडते.



`<img id="theLogo">`

नाव: theLogo

कोड

```

```

२ getElementById — नावाने शोधा

`document.getElementById("नाव")` पानावर त्या नावाची गोष्ट शोधून आपल्याला ती **ऑब्जेक्ट** म्हणून देते. आपण ती व्हेरिअबलमध्ये ठेवू शकतो, म्हणजे पुन्हा पुन्हा शोधावी लागत नाही.



पान

`getElementById("theLogo")`

our_logo

कोड

```
let our_logo = document.getElementById("theLogo");
```

३ चित्राच्या प्रॉपर्टीज

आपल्या चित्र-ऑब्जेक्टला काही प्रॉपर्टीज असतात: **src** (कोणती फाइल), **width** (रुंदी), **height** (उंची). आपण त्या फक्त वाचूही शकतो किंवा बदलूही शकतो! **src** बदलले की **दुसरे चित्र** दिसते.



our_logo

प्रॉपर्टीज

- src — फाइलीचे नाव
- width — रुंदी
- height — उंची

जादू

- (src बदला → चित्र बदलते!)

कोड

```
alert(our_logo.src);
alert(our_logo.width);
our_logo.src = "new_logo.gif"; // चित्र बदलले!
```

४ प्रयोग: चित्राचे वर्णन

describe_image() नावाचे फंक्शन बनवा जे alert मध्ये चित्राची फाइल, रुंदी आणि उंची सांगते. प्रत्येक माहिती नव्या ओळीत येण्यासाठी **\n** वापरतात.

या पानाचा संदेश

File: logo.gif
Width: 200
Height: 80

OK

कोड

```
function describe_image()
{
  let img = document.getElementById("theLogo");
  alert("File: " + img.src + "\nWidth: " + img.width + "\nHeight: " + img.height);
}
```



तुमची पाळी! करून बघा

1. **images** नावाच्या फोल्डरमधले एक चित्र पानावर लावा आणि त्याला id द्या.
2. **describe_image()** बटणावर क्लिक केल्यावर बोलवा.

★ लक्षात ठेवा

- id = चित्राला दिलेले नाव.
- document.getElementById("नाव") ने ऑब्जेक्ट मिळतो.
- img.src बदलले की चित्र बदलते.



धडा २३

साधे इमेज रोलओव्हर



तुम्ही कधी वेबसाइटवरचे बटण बघितले आहे का, जे माउस वर नेला की रंग बदलते किंवा चमकते? ती जादू म्हणजे रोलओव्हर. आज आपण ती शिकणार!

१ रोलओव्हर कसे चालते?

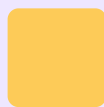
पान उघडते तेव्हा नेहमीचे चित्र दिसते. माउस चित्रावर आला की `onmouseover` चालतो आणि चित्र **दुसऱ्या चित्राने** बदलते (दोन्ही चित्रांचा आकार सारखा हवा). माउस बाहेर गेला की `onmouseout` पहिले चित्र परत लावतो.



नेहमीचे बटण



माउस वर



चमकणारे बटण



माउस बाहेर → परत

२ चित्र बदलायचा कोड

पहिल्यांदा आपण चित्राला id देतो, मग `getElementById` ने ते शोधून त्याचे `src` बदलतो.



कोड

```
let button_img = document.getElementById("button");
button_img.src = "new_image.jpg";


```

३ कोड स्वच्छ करा — फंक्शन

वरचा कोड HTML मध्ये खूप गुंतागुंतीचा दिसतो. म्हणून एक फंक्शन बनवतो जे कोणत्याही चित्राचा id आणि नवीन फाइल घेते.



कोड

```
function swap_image( id, new_image )
{
  document.getElementById(id).src = new_image;
}


```

४ this — “मी स्वतः”

हँडलर ज्या चित्रावर आहे, त्याच चित्राला बदलायचे असेल तर **this** शब्द वापरतात. **this** म्हणजे “मी स्वतः” — अवतरण चिन्हे न लावता! त्यामुळे फंक्शनला id द्यायचीही गरज राहत नाही.



चित्र

“हँडलर मलाच बोलावतो आहे — म्हणून this म्हणजे मीच!”

कोड

```
function swap_image( img, new_image )
{
  img.src = new_image;
}


```



तुमची पाली! करून बघा

- सहा बटणे (home, contacts, people, products, quotes, whatsnew) एका परिच्छेदात लावा.
- प्रत्येकावर माउस नेल्यावर `...over.jpg` चित्र येईल आणि माउस काढल्यावर मूळ चित्र परत येईल असा कोड लिहा.

★ लक्षात ठेवा

- रोलओव्हर = माउस वर आल्यावर चित्र बदलणे.
- onmouseover आणि onmouseout हँडलर्स वापरा.
- this म्हणजे ज्या टॅगमध्ये आपण आहोत तोच.



तुम्ही मित्राच्या घरी पाहुणे येणार आहेत. जर तुम्ही पाहुणे आल्यावरच सामान आणायला दुकानात गेलात तर सगळे वाट बघत राहतील! चांगला यजमान आधीच सामान आणून ठेवतो. चित्रांचेही असेच करूया — आधीच आणून ठेवू (preload).

१ रोलओव्हरमधला दोष

आपण पूर्वी केलेले रोलओव्हर चालते, पण एक अडचण आहे: माउस चित्रावर आल्यावरच ब्राउझर नवीन चित्र इंटरनेटवरून आणतो. आपल्या कॉम्प्युटरवर हे लगेच होते, पण खऱ्या इंटरनेटवर थोडा उशीर होतो, आणि बटण चमकायला वेळ लागतो.



माउस वर



चित्र आणत आहे...



उशिरा चमकते

२ उपाय: आधीच लोड करून ठेवा

पान उघडल्यावरच दुसरी चित्रे ब्राउझरच्या मेमरीत आणून ठेवली तर माउस आल्यावर चित्र लगेच दिसते. त्यासाठी JavaScript मध्ये “आभासी चित्र” बनवतात — जे पानावर दिसत नाही, पण ब्राउझर ते आणून ठेवतो.



पान उघडले



आभासी चित्रे आणून ठेवा



माउस येताच लगेच बदल

३ new Image() — नवीन ऑब्जेक्ट बनवा

new हा शब्द “नवीन ऑब्जेक्ट बनवा” असे सांगतो — आपण अरेसाठीही वापरला होता. `new Image()` एक आभासी चित्र बनवतो. त्याला **ऑब्जेक्ट व्हेरिअबल** म्हणतात कारण त्यात एक ऑब्जेक्ट आहे. त्याचे `src` ठरवले की ब्राउझर चित्र आणतो.

virtual_image



ऑब्जेक्ट व्हेरिअबल

कोड

```
function preload_images()
{
  let virtual_image = new Image();
  virtual_image.src = "new.jpg"; // ब्राउझर आताच हे चित्र आणतो
}

<body onload="preload_images();">
```

४ अनेक चित्रे — अरे आणि लूप

खूप चित्रे असतील तर त्यांची नावे अरेत ठेवा आणि लूपने प्रत्येकासाठी एक आभासी चित्र बनवा. ऑब्जेक्ट व्हेरिअबल म्हणून अरेची एक पेटीसुद्धा चालते!



नावांचा अरे

+



for लूप

→



सगळी चित्रे आधीच तयार

कोड

```
function preload_images()
{
  let files = ["contactover.jpg", "homeover.jpg", "peopleover.jpg"];
  let loaded = new Array();
  for ( let i = 0; i < files.length; i++ )
  {
    loaded[i] = new Image();
    loaded[i].src = "images/" + files[i];
  }
}
```



तुमची पाली! करून बघा

1. धडा २३ च्या सगळ्या बटणांसाठी `preload_images()` बनवा आणि `<body onload="...">` ने बोलवा.
2. हळू इंटरनेटवर फरक दिसतो का ते आपल्या मित्राला विचारा!

★ લક્ષાત ઠેવા

- ચિત્ર આધીચ આણૂન ઠેવણે = preload.
- new Image() આભાસી ચિત્ર બનવતો.
- ઍરે + લૂપ = અનેક ચિત્રે ઁકદમ.



धडा २५

ब्राउझर विंडोज उघडणे



तुम्ही घरात नवीन खोली बांधता तेव्हा तिचा आकार, दारे, खिडक्या हे सगळे ठरवता. JavaScript ने आपण नवीन विंडो उघडू शकतो आणि तिचा आकार, ठिकाण सगळे ठरवू शकतो!

१ HTML ने नवीन विंडो

साध्या HTML लिंकने नवीन विंडो उघडता येते (`target="_new"`). पण तिचा आकार, ठिकाण किंवा दुसरे काहीच आपल्या हातात नसते.

कोड

```
<a href="#" target="_new">link</a>
```

२ window.open() — पूर्ण नियंत्रण

JavaScript ची `window.open()` मेथड तीन गोष्टी घेते: ① पान कोणते उघडायचे, ② विंडोचे नाव, ③ विंडोचे गुणधर्म (रुंदी, उंची वगैरे). तिसरी गोष्ट एकाच मजकुरात, स्वल्पविरामाने जोडून लिहितात.



पान

+



नाव

+



गुणधर्म



नवीन विंडो

कोड

```
window.open( page_url, name, parameters );
```

३ खरे उदाहरण

300×300 पिक्सेलची विंडो उघडून त्यात बातम्यांचे पान दाखवूया.



bbc — 300 × 300

News page...

कोड

```
window.open("http://news.bbc.co.uk/", "bbc", "width=300,height=300");
```

४ विंडोचे गुणधर्म

हे गुणधर्म तुम्ही विंडोसाठी ठरवू शकता:

गुणधर्म	किंमत	काय करतो?
width	पिक्सेल	विंडोची रुंदी
height	पिक्सेल	विंडोची उंची
location	yes / no	पत्त्याची पट्टी दाखवायची का
menubar	yes / no	मेनू पट्टी दाखवायची का
status	yes / no	खालची स्टेटस पट्टी
resizable	yes / no	आकार बदलता येईल का
scrollbars	yes / no	स्करोल पट्टी हवी का



बोटू म्हणतो: एकदा तुम्ही एखादा गुणधर्म ठरवला की बाकी सगळे “no” होतात! उदा. फक्त "scrollbars=yes" लिहिले तर फक्त स्करोल पट्टी दिसेल.

५ विंडोचा रिमोट कंट्रोल

`window.open()` नवीन विंडोचा **ऑब्जेक्ट** परत देते. तो आपण एका व्हेरिअबलमध्ये ठेवला की त्याच्याद्वारे त्या विंडोला नंतरही सूचना देता येतात — जसे टीव्हीचा रिमोट!



नवीन विंडो



win (रिमोट)

कोड

```
let win = window.open("page.html", "", "");  
// आता win वापरून त्या विंडोवर काम करता येते
```



तुमची पाळी! करून बघा

1. `chapter_25.html` मध्ये फक्त एक लोगो आणि एक लिंक ठेवा.
2. `view_new_logo()` फंक्शन बनवा: ते 200×200 ची विंडो उघडते, तिचे नाव `RocolLogo` असते आणि त्यात `images/rocollogo.jpg` दिसते. विंडो-ऑब्जेक्ट `objLogo` मध्ये ठेवा.

★ लक्षात ठेवा

- `window.open(पान, नाव, गुणधर्म)`
- `width`, `height`, `scrollbars` वगैरे गुणधर्म ठरवता येतात.
- परत मिळालेला ऑब्जेक्ट = नवीन विंडोचा रिमोट.



धडा २६

विंडो हव्या त्या जागी उघडणे



वर्गाच्या फळ्यावर चित्र लावताना ते नेमके मध्यभागी लावायचे असते. त्यासाठी तुम्ही फळ्याची रुंदी आणि चित्राची रुंदी मोजता. स्क्रीनवर विंडो मध्यभागी आणायलाही थोडे गणित करावे लागते!

१ screen ऑब्जेक्ट

screen ऑब्जेक्ट आपल्याला वापरणाऱ्याच्या स्क्रीनची माहिती देतो.

प्रॉपर्टी

काय सांगते?

screen.width	स्क्रीनची पूर्ण रुंदी (पिक्सेल)
screen.height	स्क्रीनची पूर्ण उंची
screen.availWidth	वापरता येणारी रुंदी
screen.availHeight	टास्कबार सोडून उरलेली उंची
screen.colorDepth	स्क्रीन किती रंग दाखवू शकते

२ विंडोची जागा ठरवा

विंडो उघडताना **left** (डावीकडून किती अंतरावर) आणि **top** (वरून किती अंतरावर) सांगता येते. चल (variable) वापरले तर + वापरून सगळे एकाच मजकुरात जोडायला विसरू नका.

कोड

```
window.open("page.html", "win",
"width=200,height=200,left=" + 100 + ",top=" + 50);
```

३ मध्यभागी आणायचे गणित

स्क्रीनचा मध्य = रुंदी ÷ २ आणि उंची ÷ २. पण जर आपण विंडोचा वरचा-डावा कोपरा मध्यावर ठेवला तर कोपरा मध्यावर येतो, विंडो नाही! म्हणून डावीकडून विंडोची **अर्धी रुंदी** आणि वरून **अर्धी उंची** वजा करतात.



स्क्रीन

$$\begin{aligned} \text{left} &= \\ \text{स्क्रीन} &\div 2 - \\ \text{विंडो} &\div 2 \end{aligned}$$



मध्यभागी विंडो!

कोड

```
let win_width = 200;
let win_height = 200;
let win_left = (screen.availWidth / 2) - (win_width / 2);
let win_top = (screen.availHeight / 2) - (win_height / 2);
```

४ दोन छोटी मदतनीस फंक्शन्स

जो कोड पुन्हा पुन्हा लागणार आहे त्यासाठी फंक्शन्स बनवतात (धडा १९!). एक डावीकडे अंतर काढते, दुसरे वरचे.

कोड

```
function get_win_left( width )
{
  return (screen.availWidth / 2) - (width / 2);
}
function get_win_top( height )
{
  return (screen.availHeight / 2) - (height / 2);
}
```

तुमची पाली! करून बघा

1. धडा २५ चा लोगो आता स्क्रीनच्या बरोबबर मध्यभागी उघडा. मूळ फंक्शनमध्ये जास्त बदल न करता `get_win_left()` आणि `get_win_top()` वापरा.

★ लक्षात ठेवा

- screen ऑब्जेक्ट स्क्रीनचा आकार सांगतो.
- विंडो मध्यभागी = स्क्रीन ÷ २ – विंडो ÷ २.
- पुन्हा लागणारे गणित फंक्शनमध्ये ठेवा.



धडा २७

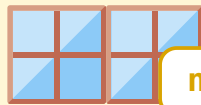
फोकस आणि ब्लर



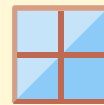
वर्गात शिक्षकांनी एका मुलाकडे बोट दाखवले की सगळ्यांचे लक्ष त्याच्याकडे जाते. विंडोलाही असे “लक्ष” (focus) मिळते. ज्या विंडोवर लक्ष असते ती सर्वांच्या पुढे येते.

१ focus() — विंडो पुढे आणा

अनेक विंडो उघड्या असतील तर `focus()` मेथड एका विंडोला **सर्वांच्या पुढे** आणते आणि ती सक्रिय करते.



विंडो एकमेकांमागे

`new_window.focus()`

ती पुढे आली!

कोड

```
new_window.focus();
```

२ blur() — लक्ष काढून घ्या

`blur()` याच्या उलट आहे — ती विंडोचे फोकस काढून घेते आणि लक्ष आधीच्या विंडोवर परत जाते.



विंडो पुढे

`.blur()`

मागे गेली

कोड

```
new_window.blur();
```

३ फोकसवर काम — onfocus

विंडोला फोकस मिळाल्यावर किंवा गेल्यावर एखादे काम करायचे असेल तर `onfocus` आणि `onblur` हँडलर वापरतात.

कोड

```
new_window.onfocus = some_function; // कंस नाही — फक्त फंक्शनचे नाव
```

४ दोन विंडोंचा प्रयोग

आपण आता दोन लोगो-विंडो उघडू: **नवा** आणि **जुना** (`oldRocolLogo`). दोन्ही विंडो एकमेकांना झाकू नयेत म्हणून वेगवेगळ्या ठिकाणी उघडायच्या.



कोड

```
objOldLogo = window.open("images/rocollogo.gif", "oldRocolLogo",  
    "width=200,height=200,left=50,top=50");
```

तुमची पाली! करून बघा

1. थडा २६ चा कोड वाढवा: दुसरी विंडो उघडा, तिचे नाव `oldRocolLogo` आणि ऑब्जेक्ट `objOldLogo` असू द्या.
2. आता `focus()` आणि `blur()` वापरून कोणती विंडो पुढे यावी ते ठरवा.

★ लक्षात ठेवा

- `focus()` विंडोला पुढे आणते.
- `blur()` विंडोचे लक्ष काढते.
- दोन विंडो वेगवेगळ्या जागी उघडाव्या.



धडा २८

वेळेवर बनणारे पान — Dynamic Content



जादूगार टोपीतून ससा काढतो. तुम्ही पण रिकाम्या विंडोत काहीही नसताना पूर्ण पान तयार करू शकता — फक्त JavaScript ने!

१ रिकामी विंडो उघडा

आधी `window.open()` मध्ये पानाचा पत्ता **रिकामा** सोडा. ब्राउझर एक पूर्ण रिकामी विंडो उघडतो. तिचा ऑब्जेक्ट व्हेरिअबलमध्ये ठेवा.



टोपी (रिकामी विंडो)



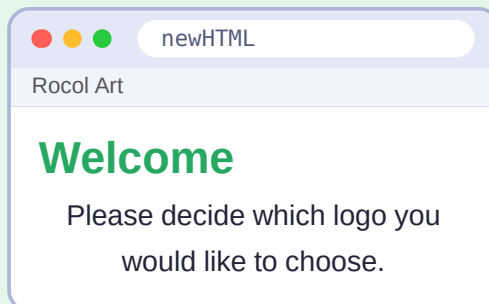
JavaScript पान बनवते

कोड

```
new_win = window.open("", "newWin", "width=400,height=400");
```

२ नव्या विंडोत लिहा

आपण आधी वापरलेले `document.write()` हे “चालू पानावर” लिहिते. पण आता आपल्याला **त्या नव्या विंडोच्या** document ला लिहायचे आहे. म्हणून `new_win.document.write(...)` असे लिहितो.

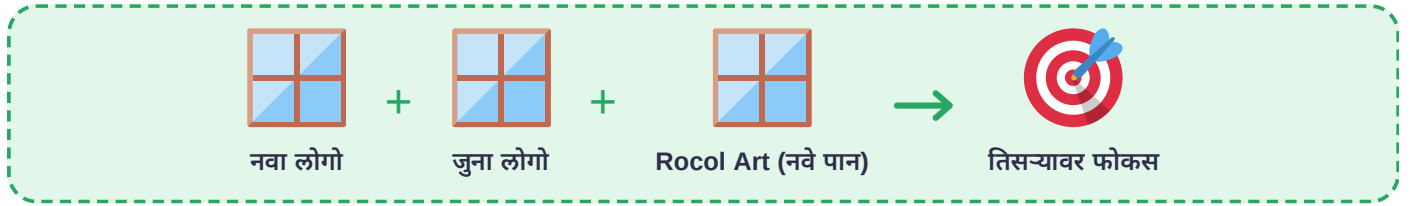
`new_win.document.write(...)`

कोड

```
new_win.document.write("<html><head><title>Rocol Art</title></head>");
new_win.document.write("<body><h1>Welcome</h1>");
new_win.document.write("<p>Please decide which logo you would like to choose.</p>");
new_win.document.write("</body></html>");
new_win.document.close(); // लिहून झाले असे सांगा
```

३ प्रयोगाचे पान

तिसऱ्या विंडोचे नाव `newHTML` आणि ऑब्जेक्ट `objNewHTML` ठेवा, आकार 400×400 , शीर्षक "Rocol Art". सगळ्या विंडो उघडल्यावर तिसऱ्या विंडोला **फोकस** द्या.



कोड

```
objNewHTML.focus();
```

तुमची पाली! करून बघा

1. रिकामी विंडो उघडून त्यात "Welcome" हे H1 शीर्षक आणि एक परिच्छेद लिहा.
2. विंडोचे शीर्षक "Rocol Art" ठेवा आणि शेवटी विंडोवर `focus()` करा.

★ लक्षात ठेवा

- `window.open("")` रिकामी विंडो उघडते.
- `नव्या_विंडो.document.write(...)` ने तिच्यात HTML लिहा.
- लिहून झाल्यावर `document.close()` करा.



आई घरात आहे आणि तिने मुलांना बाहेर खेळायला पाठवले. आईला मुलाला बोलावता येते. पण मुलाला आईशी बोलायचे असेल तर? “आई-मूल” नात्याचा उपयोग होतो! विंडोज पण अशा आई (parent) आणि मूल (child) असतात.

१ close() — विंडो बंद करा

विंडो-ऑब्जेक्ट आपल्याकडे असेल तर `close()` मेथडने ती विंडो बंद करता येते.



`new_window.close()`



विंडो बंद

कोड

```
new_window.close();
```

२ विंडोची मर्यादा (scope)

प्रत्येक विंडोचे कोड आणि व्हेरिअबल्स त्याच विंडोत राहतात. आईच्या विंडोतले फंक्शन मुलाच्या विंडोतून थेट बोलावता येत नाही. म्हणूनच दुसऱ्या विंडोत काही लिहायचे असेल तर आधी तिचे `document` ऑब्जेक्ट मिळवावे लागते.



आई-विंडो (parent)



तिची स्वतःची फंक्शन्स



मुलाचा रिमोट (ऑब्जेक्ट)

VS



मूल-विंडो (child)



तिची स्वतःची फंक्शन्स



आईच्या गोष्टी थेट दिसत नाहीत

३ opener — आईशी बोला

मुलाच्या विंडोतून आईशी बोलायला **opener** नावाचा खास ऑब्जेक्ट असतो. JavaScript ने उघडलेल्या प्रत्येक विंडोत तो तयार होतो आणि तो नेहमी **तिला उघडणाऱ्या विंडोकडे** बोट दाखवतो.

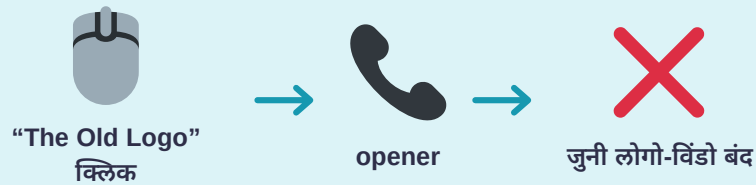


कोड

```
// मूल-विंडोतील लिंकवरून आई-विंडोतील window_2 बंद करा:  
<a href="#" onclick="opener.window_2.close();">Close window 2</a>
```

४ प्रयोग: लोगो निवडा

तिसऱ्या विंडोत दोन परिच्छेद ठेवा: “The Old Logo” आणि “The New Logo”. प्रत्येक एका लिंकमध्ये ठेवा, ज्यावर क्लिक केल्यावर आई-विंडोतील संबंधित लोगो-विंडो बंद होते.



तुमची पाळी! करून बघा

1. तिसऱ्या विंडोत दोन लिंक बनवा आणि त्यांच्या onclick मध्ये `opener.objOldLogo.close();` आणि `opener.objLogo.close();` वापरा.

★ लक्षात ठेवा

- `close()` विंडो बंद करते.
- प्रत्येक विंडोचे स्वतःचे कोड असतात.
- `opener` = आईच्या विंडोचा दुवा.



धडा ३०

बाहेरची स्क्रिप्ट फाइल



तुम्ही रोज शाळेत जाताना एकच नकाशा सगळ्यांना दाखवता का? नाही — एक नकाशा एकदाच काढता आणि सगळे तो वापरतात! तसेच आपले JavaScript एकाच फाइलमध्ये ठेवून अनेक पानांमध्ये वापरता येते.

१ एकच कोड पुन्हा पुन्हा?

समजा आपल्याकडे वर्षाची यादी बनवणारे एक फंक्शन आहे. ते अनेक पानांवर लागते. प्रत्येक पानात ते कॉपी केले तर: ① वापरणाऱ्याला तोच कोड पुन्हा पुन्हा डाउनलोड करावा लागतो. ② फंक्शन सुधारले तर **सगळ्या पानांत** बदलावे लागते.



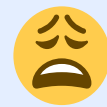
पान १
(कोडची कॉपी)



पान २
(कोडची कॉपी)



पान ३
(कोडची कॉपी)



बदल करायचा तर
सगळीकडे!

२ उपाय: एक बाहेरची फाइल

आपला सगळा JavaScript कोड **script.js** नावाच्या वेगळ्या फाइलमध्ये ठेवा आणि प्रत्येक पानात फक्त तिची **ओळख** द्या. एकदा डाउनलोड झाली की ब्राउझर ती पुन्हा आणत नाही. आणि फाइल बदलली की **सगळ्या पानांत** लगेच बदल दिसतो!



पान १



पान २



पान ३



script.js (एकच फाइल)

कोड

```
<script type="text/javascript" src="script.js"></script>
```

३ महत्वाचे नियम

src मध्ये फाइलचा पत्ता लिहितात — चित्र लावतो तसेच. तीन गोष्टी लक्षात ठेवा: ① `</script>` हा शेवटचा टॅग लिहिणे **आवश्यक** आहे. ② अशा टॅगच्या आत आपण वेगळा JavaScript कोड लिहू शकत नाही. ③ आत्ताच्या पानासाठी कोड हवा असेल तर दुसरा वेगळा `<script>` टॅग वापरा.

`<script
src="script.js">` `</script>`



यात वेगळे काही लिहू
नका

कोड

```
<script src="script.js"></script>      <!-- बाहेरची फाइल -->
<script>
  alert("हा या पानापुरता कोड");        // वेगळ्या टॅगमध्ये
</script>
```



तुमची पाळी! करून बघा

- तुमची सगळी फंक्शन्स `script.js` मध्ये हलवा.
- `head` मधल्या script टॅगमध्ये `src="script.js"` वापरा आणि पान अजूनही चालते का तपासा.

★ लक्षात ठेवा

- कोड बाहेरच्या `.js` फाइलमध्ये ठेवता येतो.
- `<script src="..."></script>` ने तिला जोडतात.
- एकदा बदल = सगळ्या पानांत बदल.



शाळेत प्रवेशाचा अर्ज भरल्यावर कारकून तो तपासतो आणि चूक असली तर परत देतो. पण जर अर्ज द्यायच्या आधीच तुमचा मित्र तो तपासून देईल तर? JavaScript फॉर्मसाठी तोच मित्र आहे!

१ फॉर्म आणि JavaScript

JavaScript नसेल तर फॉर्मची सगळी तपासणी **सर्व्हर** करतो — म्हणजे चूक असली तर वापरणाऱ्याला वाट बघावी लागते.

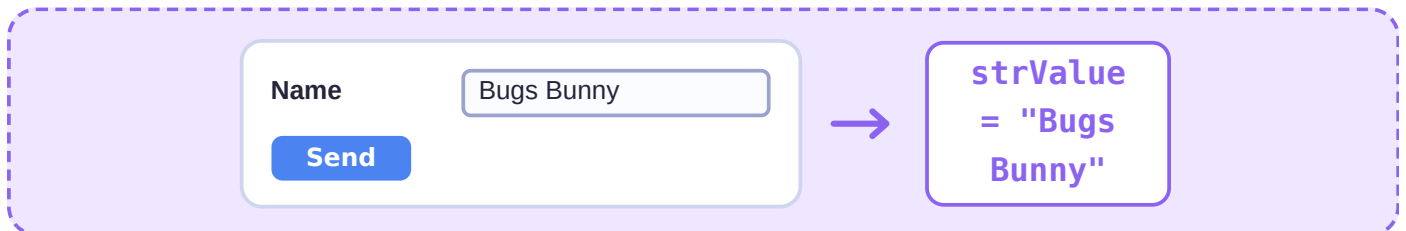
JavaScript वापरली की चुका आपल्या ब्राउझरमध्येच पकडता येतात आणि फक्त **बरोबर माहिती** सर्व्हरकडे जाते. पण लक्षात ठेवा:

JavaScript सर्व्हरच्या तपासणीला **पर्याय नाही**, तिला मदतनीस आहे.



२ फॉर्ममधली माहिती वाचा

फॉर्मला id द्या आणि `getElementById` ने तो शोधा. मग फॉर्मच्या आतल्या टेक्स्ट बॉक्सचे नाव लिहून `.value` लिहिले की वापरणाऱ्याने लिहिलेला मजकूर मिळतो!



कोड

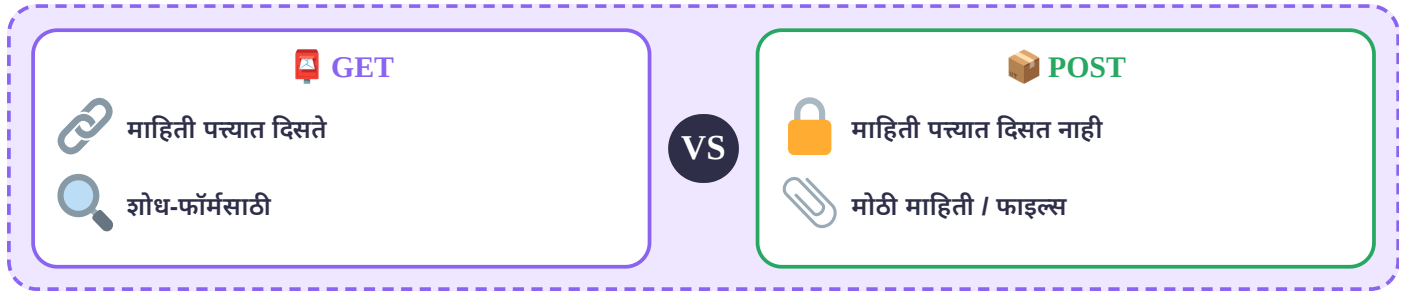
```

<form id="Enquiry" method="GET">
  <input type="text" name="Name">
</form>

let objForm = document.getElementById("Enquiry");
let strValue = objForm.Name.value;
  
```

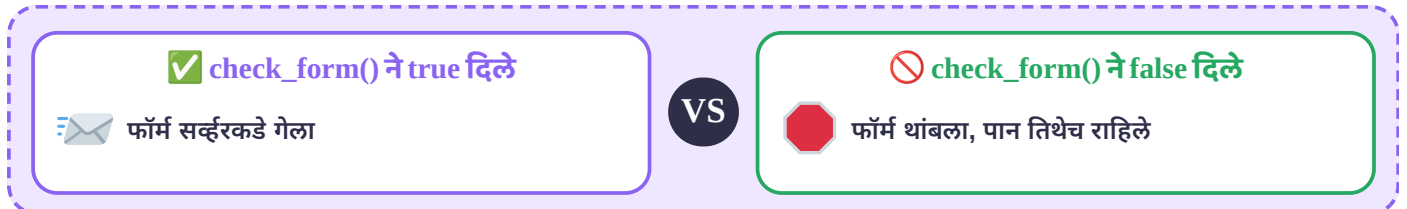
३ माहिती पाठवण्याचे दोन मार्ग: GET आणि POST

फॉर्मची माहिती सर्व्हरला दोन प्रकारे पाठवता येते. **GET**: माहिती पत्त्यातच दिसते — शोध-फॉर्मसाठी चांगले. **POST**: माहिती आतून पाठवली जाते — जास्त माहिती (फाइल, मोठा मजकूर) असेल तेव्हा.



४ सबमिट थांबवा — onsubmit आणि return false

फॉर्म “पाठवा” (submit) केला की ब्राउझर थांबून हँडलरचे उत्तर पाहतो. हँडलरने **true** दिले तर फॉर्म पाठवला जातो, **false** दिले तर पाठवला जात नाही. म्हणून हँडलर असा लिहितात: `onsubmit="return check_form();"`



कोड

```
<form id="Enquiry" onsubmit="return check_form();" >
```

५ प्रयोग: “Bugs Bunny” चेक

`check_form()` फंक्शन बनवा: नावाचा बॉक्स वाचा. नाव “Bugs Bunny” नसेल तर alert दाखवा आणि **false** परत द्या, नाहीतर **true**.

या पानाचा संदेश

Sorry chum! Either you misspelled your name... Or you haven't got what it takes... Try again.

OK

कोड

```
function check_form()
{
  let objForm = document.getElementById("Enquiry");
  let name = objForm.Name.value;
  if ( name != "Bugs Bunny" )
  {
    alert("Sorry chum! Either you misspelled your name...\n" +
      "Or you haven't got what it takes... Try again.");
    return false;
  }
  return true;
}
```



तुमची पाली! करून बघा

1. नाव आणि वय असलेला फॉर्म बनवा आणि `check_form()` ने तपासा.
2. कोड ठीक चालतोय ना हे बघायला `return false` काढून बघा — काय बदल होतो?

★ लक्षात ठेवा

- JavaScript फॉर्म पाठवण्याआधी तपासते.
- `objForm.Name.value` ने टेक्स्ट बॉक्समधला मजकूर मिळतो.
- `onsubmit="return ..."` आणि `false` = फॉर्म थांबतो.



फॉर्ममधले प्रत्येक बटण, बॉक्स किंवा मेनू हे एक ऑब्जेक्ट आहे — म्हणजे त्याच्या स्वतःच्या प्रॉपर्टीज, मेथड्स आणि इव्हेंट्स असतात. चला त्यांची ओळख करून घेऊ!

१ प्रत्येक वस्तूची स्वतःची प्रॉपर्टी

फॉर्मच्या प्रत्येक गोष्टीला (टेक्स्ट बॉक्स, बटण वगैरे) काही प्रॉपर्टीज असतात. सर्वात उपयोगी म्हणजे **value** — बॉक्समधली किंमत. ती वाचता येते आणि बदलताही येते.



टेक्स्ट बॉक्स (Name)

गुणधर्म (Properties) — विशेषणे

- value — आत लिहिलेला मजकूर
- name — त्याचे नाव

मेथड्स

- focus() — लक्ष द्या
- blur() — लक्ष काढा
- select() — मजकूर निवडा

कोड

```
let v = document.getElementById("Enquiry").Name.value;
document.getElementById("Enquiry").Name.value = "please try again";
```

२ महत्वाच्या मेथड्स

या मेथड्स फॉर्मच्या गोष्टींना सूचना देतात:

मेथड	काय करते?
<code>text.focus()</code>	त्या बॉक्सवर कर्सर नेते
<code>text.blur()</code>	बॉक्सवरून कर्सर काढते
<code>text.select()</code>	बॉक्समधला मजकूर निवडते (हायलाइट)
<code>form.submit()</code>	फॉर्म पाठवते
<code>form.reset()</code>	फॉर्मच्या सगळ्या गोष्टी पूर्वस्थितीत आणते

३ इव्हेंट हँडलर्स

टेक्स्ट बॉक्ससाठी हे इव्हेंट्स वापरतात:

हँडलर	केव्हा चालतो?
onfocus	बॉक्सवर कर्सर आला
onblur	बॉक्समधून कर्सर बाहेर गेला
onselect	वापरणाऱ्याने काही मजकूर निवडला
onchange	मजकूर बदलला आणि बॉक्स सोडला



बोटू म्हणतो: बटणासाठी **onclick**, **onfocus** आणि **onblur** हे हँडलर्स उपयोगी आहेत.

४ प्रयोग: जास्त हुशार `check_form()`

“Bugs Bunny” लिहिले तर `submit()` ने फॉर्म स्वतः पाठवा. नाहीतर: ① `alert` ② बॉक्समध्ये “please try again” लिहा ③ बॉक्सवर **focus** आणा ④ तो मजकूर **select** करा.



चुकीचे नाव



`alert`



“please try again”



`focus + select`

कोड

```
function check_form()
{
  let f = document.getElementById("Enquiry");
  if ( f.Name.value == "Bugs Bunny" )
  {
    f.submit();
  }
  else
  {
    alert("Sorry chum! Try again.");
    f.Name.value = "please try again";
    f.Name.focus();
    f.Name.select();
  }
}
```



तुमची पाळी! करून बघा

- धडा ३१ चे `check_form()` वरीलप्रमाणे सुधारा.
- बॉक्सवर कर्सर आला की निळी पार्श्वभूमी करणारा `onfocus` हँडलर बनवून बघा.

★ लक्षात ठेवा

- value ही सर्वात उपयोगी प्रॉपर्टी आहे.
- focus(), blur(), select(), submit(), reset() मेथड्स.
- onfocus, onblur, onchange... हँडलर्स.



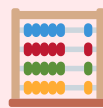
JavaScript मध्ये एक कॅल्क्युलेटर आधीपासूनच ठेवलेला असतो! त्याचे नाव आहे Math. त्याच्या मेथड्स वापरून आपण पूर्णांक काढणे, मोठा आकडा शोधणे, वर्गमूळ काढणे असे सगळे करू शकतो.

१ Math — तयार कॅल्क्युलेटर

Math हा JavaScript चा तयार ऑब्जेक्ट आहे. त्याच्या मेथड्स गणिते सोडवतात. अगदी वापरून बघा!

1 2
3 4

Math



Math.round(2.6)



3



बोद्ध म्हणतो: जुन्या पुस्तकात `Math.ceil()` चुकून `Math.cell()` असे छापले आहे. बरोबर नाव **ceil** आहे (ceiling = छत).

२ Math च्या उपयोगी मेथड्स

या मेथड्स सगळ्यात जास्त वापरतात:

मेथड	काय देते?	उदाहरण
<code>Math.ceil(x)</code>	वर पूर्णांक	<code>ceil(2.2) → 3</code>
<code>Math.floor(x)</code>	खाली पूर्णांक	<code>floor(2.6) → 2</code>
<code>Math.round(x)</code>	जवळचा पूर्णांक	<code>round(2.6) → 3</code>
<code>Math.max(a, b)</code>	दोघांतले मोठे	<code>max(4, 9) → 9</code>
<code>Math.min(a, b)</code>	दोघांतले लहान	<code>min(4, 9) → 4</code>
<code>Math.sqrt(x)</code>	वर्गमूळ	<code>sqrt(16) → 4</code>
<code>Math.random()</code>	0 ते 1 मधली यादृच्छिक संख्या	<code>random() → 0.5731...</code>

३ ceil आणि floor — संख्या-रेषा

ceil म्हणजे “छत” — संख्या वर ढकलतो. **floor** म्हणजे “जमीन” — संख्या खाली आणतो. 2.2 आणि 2.6 दोन्हीला $\text{ceil} = 3$ आणि $\text{floor} = 2$.



$\text{floor}(2.6) = 2$

2.6



$\text{ceil}(2.2) = 3$

कोड

```
Math.ceil(2.2);    // 3
Math.floor(2.6);   // 2
Math.round(2.6);   // 3
```

४ सर्वरशिवाय फॉर्म

तुम्हाला फॉर्ममधली माहिती सर्वरला पाठवायचीच नाही — फक्त हिशोब करायचा आहे? मग “Submit” बटण नको आणि **action / method** पण नको. फक्त इव्हेंट हँडलर्स वापरून JavaScript ला काम द्या!

Player 1

7

Player 2

4

Winner

Player 1

Find Winner



Math.max ने जिंकणारा

कोड

```
function find_winner()
{
  let f = document.getElementById("MaxWins");
  let a = parseInt(f.Player1.value);
  let b = parseInt(f.Player2.value);
  let big = Math.max(a, b);
  if ( a == b )      f.Winner.value = "Draw";
  else if ( big == a ) f.Winner.value = "Player 1";
  else              f.Winner.value = "Player 2";
}
```



बोट्टू म्हणतो: `parseInt()` बॉक्समधल्या मजकुराला खऱ्या संख्येत बदलते. नाहीतर `"7" + "4"` हे `"74"` होईल, 11 नाही!



तुमची पाळी! करून बघा

- दोन खेळाडू आपापला आकडा लिहितात. जास्त आकडा असलेला जिंकतो! असा “Max Wins” गेम बनवा.
- `Math.random()` वापरून 1 ते 6 पर्यंत फासा फेकणारे बटण बनवा: `Math.floor(Math.random() * 6) + 1`.

★ लक्षात ठेवा

- Math ऑब्जेक्टच्या मेथड्स गणिते करतात.
- ceil वर, floor खाली, round जवळच्या पूर्णांकाकडे.
- सर्व्हर नसेल तरी JavaScript फॉर्मचे हिशोब करू शकते.



वर्गात एखाद्या मित्राला रोज पूर्ण नावाने “अर्जुन विश्वनाथ देशमुख” असे हाक मारायची का? नाही! आपण “अर्जुन” म्हणतो — छोटे नाव! कोडमध्येही लांब नावांना छोटी नावे देता येतात.

१ लांब नावांचा त्रास

फॉर्ममधल्या एका बॉक्सला पूर्ण नावाने बोलवायचे तर असे लिहावे लागते — हे लिहिताना चुकाही होऊ शकतात आणि वेळ जातो.

`document.getElementById("MaxWins").Player1`



खूप लांब!

कोड

```
document.getElementById("MaxWins").Player1.value
```

२ उपाय: ऑब्जेक्ट व्हेरिअबल

ज्या ऑब्जेक्टचा आपण वारंवार वापर करतो त्याला एक छोटे नाव द्या — म्हणजे ऑब्जेक्ट एका व्हेरिअबलमध्ये ठेवा. मग त्या छोट्या नावानेच प्रॉपर्टीज वापरा. (नावाच्या आधी `o` लावणे ही फक्त सवय आहे — ती ऑब्जेक्ट व्हेरिअबल ओळखायला मदत करते.)



`oPlayer1`

=



Player1 चा बॉक्स



`oPlayer1.value`

कोड

```
let oPlayer1 = document.getElementById("MaxWins").Player1;
alert( oPlayer1.value ); // आता कमी लिहावे लागते
```

३ एक सावधानता

व्हेरिअबलमध्ये ऑब्जेक्ट ठेवा, त्याची किंमत नाही. जर तुम्ही `.value` पर्यंत लिहिले तर व्हेरिअबलमध्ये फक्त मजकूर जातो आणि ऑब्जेक्टची जादू (मेथड्स, बदल) हरवते.

✓ बरोबर



`oPlayer1 = ...Player1`



`oPlayer1.value, .focus()` सगळे चालते

VS

✗ चूक



`x = ...Player1.value`



`x` मध्ये फक्त मजकूर, `.focus()` चालत नाही



तुमची पाली! करून बघा

1. धडा ३३ चा Max Wins कोड ऑब्जेक्ट ढेरिअबल्स वापरून लहान करा: `oPlayer1` , `oPlayer2` , `oWinner` .

★ लक्षात ठेवा

- वारंवार वापरायच्या ऑब्जेक्टला छोटे नाव द्या.
- ऑब्जेक्ट ठेवा, त्याची `.value` नाही.
- नावाआधी `o` लावणे ही फक्त सवय.



धडा ३५

मेनूमधून कृती



हॉटेलच्या मेनूतून तुम्ही एक पदार्थ निवडता आणि वेटर लगेच तो आणतो. वेबपानावरच्या मेनूतून (ड्रॉप-डाउन) एखादा पर्याय निवडला की आपण लगेच काहीतरी करू शकतो!

१ select — मेनू बनवा

HTML मध्ये `<select>` आणि `<option>` ने मेनू बनतो. प्रत्येक पर्यायाला एक **value** असते — ती निवडल्यावर मिळते.

Image one ▼

Select a destination

Image one

Image two

Image three

कोड

```
<form id="menu">
  <select name="Product">
    <option value="one">Image one</option>
    <option value="two">Image two</option>
    <option value="three">Image three</option>
  </select>
</form>
```

२ निवडल्यावर काम — onchange

सामान्यपणे मेनूतून पर्याय निवडला की फक्त **value** बदलते. आपल्याला त्याच क्षणी काहीतरी करायचे असेल तर मेनूचा **onchange** हँडलर वापरतात. तो मेनूची किंमत बदलली की चालतो.



मेनूतून निवड



onchange



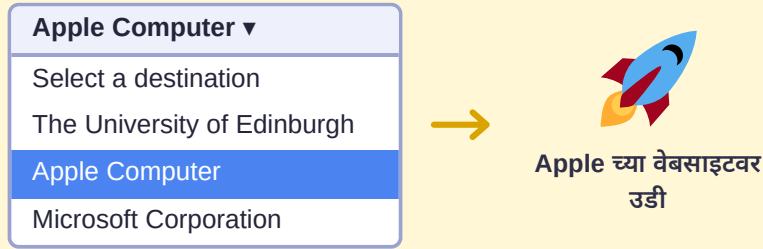
आपले फंक्शन



बोद्द म्हणतो: मेनूसाठी इव्हेंट्स: `onfocus`, `onblur`, `onchange`. आणि मेथड्स: `focus()`, `blur()`.

३ प्रयोग: उडी मारणारा मेनू

मेनूतून एखादी वेबसाइट निवडा आणि पान आपोआप तिथे जाईल! सगळ्यात वरचा पर्याय "Select a destination" असतो आणि त्याची value रिक्ामी ("") असते — त्याला निवडले तर काहीच होत नाही.



कोड

```
<select name="Destination" onchange="jump_to_page();">
  <option value="">Select a destination</option>
  <option value="http://www.ed.ac.uk/">The University of Edinburgh</option>
  <option value="http://www.apple.com/">Apple Computer</option>
  <option value="http://www.microsoft.com/">Microsoft Corporation</option>
</select>

function jump_to_page()
{
  let oMenu = document.getElementById("menu"); // फॉर्म
  let url = oMenu.Destination.value;           // निवडलेली value
  if ( url != "" )
  {
    document.location = url;
  }
}
```

🔧 तुमची पाळी! करून बघा

- वरचा "उडी मारणारा मेनू" तुमच्या आवडत्या तीन वेबसाइट्ससाठी बनवा.
- वापरणाऱ्याला आधी सावध करण्यासाठी `confirm()` पण जोडा.

★ लक्षात ठेवा

- `<select>` आणि `<option>` ने मेनू बनतो.
- `onchange` = निवड बदलली की चालतो.
- `menu.Name.value` ने निवडलेली value मिळते.

३६

धडा ३६

फॉर्ममध्ये उत्तर आवश्यक करणे



प्रश्नपत्रिकेत शिक्षक सांगतात, “सगळ्या प्रश्नांची उत्तरे द्या!” कोणी एखादे उत्तर सोडले तर शिक्षक ते लगेच सांगतात. आपण पण फॉर्ममध्ये सगळे आवश्यक भाग भरले आहेत का ते तपासू शकतो.

१ का तपासायचे?

फॉर्ममध्ये जर आवश्यक माहिती नसेल तर तो सर्व्हरला पाठवण्यात काहीच अर्थ नाही — वापरणाऱ्याचा आणि सर्व्हरचा वेळ वाया जातो. त्यापेक्षा पाठवण्याआधीच तपासा आणि चूक दाखवा.



अपूर्ण फॉर्म



JavaScript थांबवते



“कृपया पर्याय निवडा”

२ मेनूमध्ये निवड झाली का?

मेनूचा पहिला पर्याय (डिफॉल्ट) जर **रिकामी value** ("") असलेला ठेवला, तर वापरणाऱ्याने काहीच निवडले नाही तर तीच value मिळते. आपण ती "" शी तुलना करून चूक पकडतो.

Please select a product ▼



```
value ==
""
```



या पानाचा संदेश

please select a product

OK

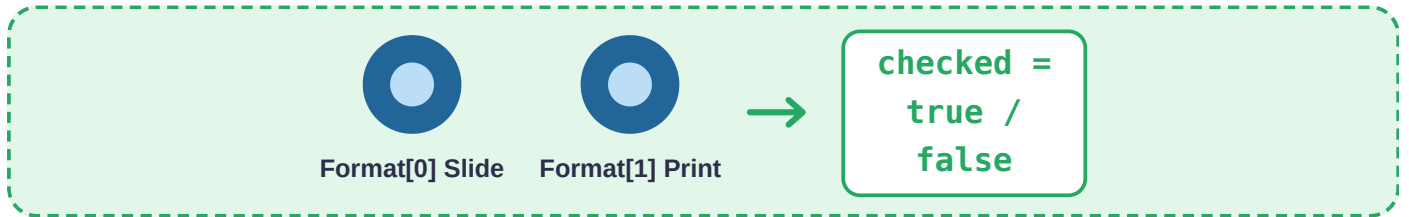
कोड

```
if ( oOrderForm.Product.value == "" )
{
    alert("please select a product");
    return false;
}
```

३ रेडिओ बटणे — एकच निवडा

रेडिओ बटणांच्या गटाला एकच नाव असते. त्यामुळे नावाने एक बटण निवडता येत नाही, म्हणून अर्रसारखे क्रमांक वापरतात:

`Format[0]`, `Format[1]`. प्रत्येकाची **checked** प्रॉपर्टी `true` किंवा `false` असते. कमीत कमी एक तरी `checked` आहे का ते लूपने तपासतात.



कोड

```
let found = false;
for ( let i = 0; i < oOrderForm.Format.length; i++ )
{
    if ( oOrderForm.Format[i].checked ) { found = true; }
}
if ( !found ) { alert("please choose a format"); }
```

४ मेनू निवडल्यावर रेडिओ आपोआप निवडा

काही पदार्थ फक्त “Slide” स्वरूपातच मिळतात असे समजा. मेनूच्या `onchange` मध्ये रेडिओची **checked** प्रॉपर्टी `true` करता येते.

कोड

```
function SetFormat()
{
    let oOrderForm = document.getElementById("OrderForm");
    if ( oOrderForm.Product.value == "Greek Boat" )
    {
        oOrderForm.Format[0].checked = true;
    }
}
```

५ सगळ्या चुका एकदम दाखवा

चुका एकेक करून दाखवणे वापरणाऱ्याला कंटाळवाणे वाटते. त्यापेक्षा **सगळ्या चुका एका संदेशात** दाखवा. व्हेरिअबल `error` रिकामा ठेवा आणि प्रत्येक चूक `+=` ने जोडा (`\n` नवी ओळ). शेवटी `error` रिकामा नसेल तर संदेश दाखवा आणि `false` परत द्या. फॉर्म पूर्वस्थितीत आणायला `form.reset()` वापरतात.

या पानाचा संदेश

- please select a product
- please choose a format

OK

कोड

```
function check_form()
{
  let error = "";
  if ( oOrderForm.Product.value == "" ) { error += "please select a product\n"; }
  // ... रेडिओ तपासा, पुन्हा error += ...
  if ( error != "" )
  {
    alert(error);
    return false;
  }
  return true;
}
```

तुमची पाळी! करून बघा

1. एक “ऑर्डर फॉर्म” बनवा — मेनू, रेडिओ बटणे आणि नाव. वरच्या सगळ्या तपासण्या एकाच `check_form()` मध्ये करा.
2. सगळ्या चुका एकदम दाखवा आणि “Reset” बटणाने फॉर्म साफ करा.

★ लक्षात ठेवा

- रिकामी value ("") असलेला पहिला पर्याय = “निवडच नाही”.
- रेडिओ बटणे अॅरसारखी; checked प्रॉपर्टी तपासा.
- सगळ्या चुका एकत्र दाखवण्यासाठी `error += ...` वापरा.



कॅलेंडर आणि घड्याळ दोन्ही तुमच्या कॉम्प्युटरमध्ये असते. JavaScript मध्ये त्याला विचारायचे — “आज कोणती तारीख आहे?” — आणि उत्तर मिळते!

१ Date ऑब्जेक्ट

Date ऑब्जेक्ट तारीख आणि वेळ सगळे साठवतो — वर्षापासून मिलिसेकंदांपर्यंत. नवीन Date बनवायला **new Date()** लिहितात. कंसात काहीच नाही दिले तर तो **आत्ताची** तारीख आणि वेळ घेतो.



कोड

```
let dtNow = new Date();
```

२ स्वतःची तारीख ठरवा

तारीख हवी तशी ठरवता येते. क्रम: **वर्ष, महिना, दिवस, तास, मिनिटे, सेकंद, मिलिसेकंद**.

कोड

```
let theDate = new Date( 2000, 9, 10, 21, 15, 0, 0 ); // 10 ऑक्टोबर 2000, रात्री 9:15
```



बोटू म्हणतो: सावधान! महिने 0 ते 11 असे मोजतात — जानेवारी = 0, ऑक्टोबर = 9! पण दिवस 1 ते 31 असतात.

३ महिने कसे मोजतात?

लक्षात ठेवा, JavaScript मध्ये मोजणी 0 पासून — आपण धडा १४ मध्ये बघितले होते!

0	1	2	3	4	5	6	7	8	9	10	11
जाने	फेब्रु	मार्च	एप्रिल	मे	जून	जुलै	ऑगस्ट	सप्टें	ऑक्टो	नोव्हें	डिसें

४ मजकुरातून तारीख

तारीख मजकूर म्हणूनही लिहिता येते. तास पुढचे भाग ऐच्छिक आहेत. इतकेच पुरेसे:

कोड

```
let theDate = new Date("Sun, 10 Oct 2000 21:15:00");
let theDate2 = new Date("10 Oct 2000"); // हे सुद्धा चालते
```

५ प्रयोग: आजची तारीख alert मध्ये

head मध्ये असे फंक्शन बनवा जे alert मध्ये **आत्ताची तारीख आणि वेळ** दाखवते, आणि body च्या script मधून त्याला बोलवा.

या पानाचा संदेश

Thu Oct 08 2026 10:31:07

OK

कोड

```
function show_date()
{
    alert( new Date() );
}
// body मध्ये:
show_date();
```



तुमची पाळी! करून बघा

1. तुमचा वाढदिवस Date ऑब्जेक्टमध्ये ठेवून alert मध्ये दाखवा. महिना बरोबर लिहिला का ते तपासा!
2. आत्ताची तारीख आणि वेळ पानावर `document.write` ने छपा.

★ लक्षात ठेवा

- `new Date()` आत्ताची तारीख-वेळ देते.
- महिने 0 ते 11, दिवस 1 ते 31.
- तारीख मजकुरातूनही बनवता येते.



धडा ३८

Date मधून माहिती काढणे



Date ऑब्जेक्ट म्हणजे एक पेटी आहे ज्यात तारीख आणि वेळ ठेवलेली आहे. आता त्या पेटीतून हवा तो भाग — फक्त वर्ष, फक्त तास — बाहेर काढायला शिकूया!

१ Date च्या “get” मेथड्स

या मेथड्स Date मधून एकेक भाग काढतात:

मेथड	काय देते?
<code>getDate()</code>	महिन्यातला दिवस (1–31)
<code>getDay()</code>	आठवड्यातला वार (0 = रविवार ... 6 = शनिवार)
<code>getMonth()</code>	महिना (0–11)
<code>getFullYear()</code>	चार अंकी वर्ष
<code>getHours()</code>	तास (0–23)
<code>getMinutes()</code>	मिनिटे (0–59)
<code>getSeconds()</code>	सेकंद (0–59)
<code>getMilliseconds()</code>	मिलिसेकंद (0–999)
<code>getTime()</code>	1 जानेवारी 1970 पासूनचे मिलिसेकंद

२ `getTime()` — सगळे मिलिसेकंदांत

JavaScript तारीख-वेळ आत **एक मोठी संख्या** म्हणून साठवते — 1 जानेवारी 1970 पासून किती मिलिसेकंद झाले. ही संख्या दोन तारखांमधले अंतर काढायला खूप उपयोगी आहे.



कोड

```
let dtNow = new Date();
document.write( dtNow.getTime() );
```

३ दोन तारखांमधले दिवस

दोन तारखांचे मिलिसेकंद वजा करा. मग दिवस मिळवण्यासाठी भागा: $\div 1000$ (सेकंद) $\div 60$ (मिनिटे) $\div 60$ (तास) $\div 24$ (दिवस). शेवटी `Math.floor()` ने पूर्ण दिवस घ्या.

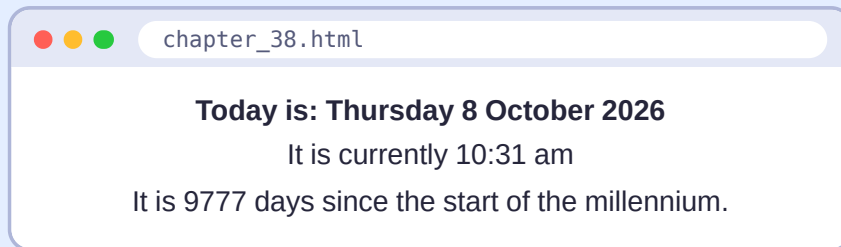


कोड

```
let start = new Date(2000, 0, 1); // 1 जानेवारी 2000
let now = new Date();
let ms = now.getTime() - start.getTime();
let days = Math.floor( ms / 1000 / 60 / 60 / 24 );
```

४ प्रयोग: तारीख आणि वेळ सुंदर दाखवा

पानावर लिहा: "Today is: ...", खाली "It is currently hh:mm am/pm", आणि "It is n days since the start of the millennium." 24 तासांच्या घड्याळातून 12 तासांत जाण्यासाठी 12 पेक्षा जास्त असलेल्या तासातून 12 वजा करा, 0 असेल तर 12 लिहा, आणि 12 किंवा जास्त असेल तर "pm" लावा.



कोड

```
let days = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];
let months = ["January", "February", "March", "April", "May", "June", "July",
  "August", "September", "October", "November", "December"];
let d = new Date();
document.write("Today is: " + days[d.getDay()] + " " + d.getDate() + " " +
  months[d.getMonth()] + " " + d.getFullYear() + "<br>");

let h = d.getHours();
let suffix = (h >= 12) ? "pm" : "am";
h = h % 12; if ( h == 0 ) { h = 12; }
let m = d.getMinutes(); if ( m < 10 ) { m = "0" + m; }
document.write("It is currently " + h + ":" + m + " " + suffix);
```

तुमची पाली! करून बघा

1. वरचा कोड बनवा आणि तिसरी ओळ पूर्ण करा: सन 2000 पासून किती दिवस झाले?
2. तुमच्या पुढच्या वाढदिवसाला किती दिवस राहिले ते काढा.

★ लक्षात ठेवा

- getDate, getMonth, getFullYear, getHours... = भाग काढतात.
- getTime() = 1970 पासूनचे मिलिसेकंद.
- दोन तारखा वजा केल्या की अंतर मिळते.



धडा ३९

JavaScript चे घड्याळ बनवूया



हा आपला शेवटचा धडा! आपण स्वतःचे चालणारे डिजिटल घड्याळ बनवणार आहोत — जे प्रत्येक सेकंदाला आपोआप बदलते. आतापर्यंत शिकलेले सगळे वापरू!

१ setTimeout — “नंतर कर”

`setTimeout()` एखादे काम **ठरावीक वेळानंतर** (मिलिसेकंदांत) चालवते. 1000 मिलिसेकंद = 1 सेकंद. जणू तुम्ही अलार्म लावला: “१ सेकंदानंतर हे काम कर!”



अलार्म लावा



१ सेकंद वाट बघा



फंक्शन चालते

कोड

```
window.setTimeout("functionName()", 1000);
```



बोद्ध म्हणतो: जुन्या पद्धतीत फंक्शनचे नाव अवतरण चिन्हात लिहितात. आजकाल `setTimeout(functionName, 1000)` असे अवतरणाशिवाय लिहितात — दोन्ही चालते.

२ अलार्म रद्द करा — clearTimeout

अलार्मला एक नाव (व्हेरिअबल) दिले तर तो गरज पडल्यास **रद्द** करता येतो. उदाहरण: “५ सेकंदांनी विंडो बंद कर — पण बटण दाबले तर नको.”



अलार्म



बटण दाबले



clearTimeout

कोड

```
firstTimeout = window.setTimeout("window.close()", 5000);
```

```
// बटणाच्या onclick मध्ये:
```

```
window.clearTimeout(firstTimeout);
```

३ फंक्शन स्वतःलाच बोलावते!

सहसा फंक्शनने स्वतःलाच बोलावणे वाईट असते (असंख्य संदेश येत राहतील!). पण **setTimeout** च्या मदतीने फंक्शन दर सेकंदाला स्वतःला परत बोलावू शकते — आणि तेच तर आपले घड्याळ!



वेळ घ्या



दाखवा



0.5 सेकंदाने स्वतःला
पुन्हा बोलवा



...

४ 0 जोडायची युक्ती

11:00:05 ऐवजी 11:0:5 दिसले तर वाईट दिसते! म्हणून संख्या 10 पेक्षा लहान असेल तर तिच्यापुढे "0" जोडतात — ternary ऑपरेटरने (धडा १३).

5



(5 < 10
) ? "0" +
 5 : 5



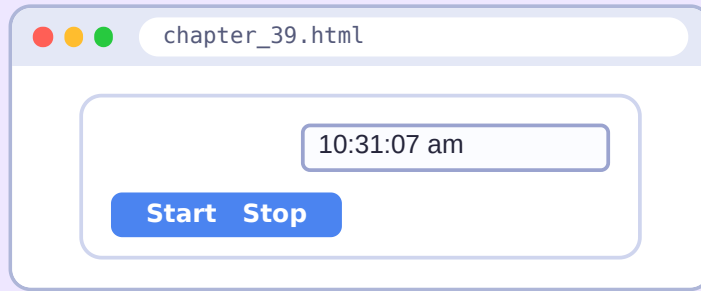
"05"

कोड

```
let s = theDate.getSeconds();  
s = ( s < 10 ) ? "0" + s : s;
```

५ घड्याळाचा पूर्ण कोड

दोन फंक्शन्स: **start_clock()** आणि **stop_clock()**. आणि दोन व्हेरिअबल्स: **timer** (अलार्म) आणि **timer_running** (घड्याळ चालू आहे का). घड्याळाचा वेळ **clockBox** नावाच्या बॉक्समध्ये दिसतो.



कोड

```
let timer;
let timer_running = false;

function start_clock()
{
    let d = new Date();
    let h = d.getHours(), m = d.getMinutes(), s = d.getSeconds();
    let suffix = ( h >= 12 ) ? "pm" : "am";
    h = h % 12; if ( h == 0 ) { h = 12; }
    m = ( m < 10 ) ? "0" + m : m;
    s = ( s < 10 ) ? "0" + s : s;

    document.getElementById("clockBox").value = h + ":" + m + ":" + s + " " + suffix;
    timer = window.setTimeout("start_clock()", 500); // आणखी अर्ध्या सेकंदाने
    timer_running = true;
}

function stop_clock()
{
    if ( timer_running )
    {
        window.clearTimeout(timer);
        timer_running = false;
    }
}

// <input type="text" id="clockBox">
// <input type="button" value="Start" onclick="start_clock();">
// <input type="button" value="Stop" onclick="stop_clock();">
```

६ बॉक्सऐवजी साधा मजकूर — innerHTML

टेक्स्ट बॉक्स ऐवजी `<span>` किंवा `<p>` सारख्या टॅगमध्येही वेळ दाखवता येते. अशा टॅगला **value** नसते, पण **innerHTML** नावाची प्रॉपर्टी असते — तिच्यात त्या टॅगच्या आतला मजकूर असतो.



`<input>` → `.value`



`<span>` →
`.innerHTML`

कोड

```
// आधी:  
clk = document.getElementById("clockBox");  
clk.value = formatted_time;  
  
// आता:  
clk = document.getElementById("clockBox"); // <span id="clockBox"></span>  
clk.innerHTML = formatted_time;
```



तुमची पाली! करून बघा

- घड्याळ पूर्ण बनवा आणि Start / Stop बटणे लावा.
- बटणाऐवजी `<span>` आणि `innerHTML` वापरून बघा. मग घड्याळाचा रंग आणि आकार CSS ने सुंदर करा!
- स्वतःचे आवडते बदल जोडा — आणि मित्रांना दाखवा!

★ लक्षात ठेवा

- `setTimeout` = ठरावीक वेळाने काम; `clearTimeout` = रद्द.
- फंक्शन `setTimeout` ने स्वतःला बोलावले की घड्याळ चालते.
- साध्या टॅगमधला मजकूर बदलायला `innerHTML` वापरतात.

abc शब्दांची छोटी डिक्शनरी

program	सूचनांची यादी जी कॉम्प्युटर वाचतो आणि पाळतो
browser	इंटरनेट पाहण्याचे ॲप (Chrome, Edge, Firefox)
client / server	क्लायंट = तुम्ही आणि तुमचा ब्राउझर; सर्व्हर = वेबसाइटचा मोठा कॉम्प्युटर
script	पानातच लिहिलेला छोटा प्रोग्राम
object	वस्तू — उदा. पियानो, चित्र, विंडो
property	वस्तूचा गुण — उदा. रंग, रुंदी
method	वस्तू करू शकते ती क्रिया — उदा. play()
variable	नाव दिलेला डबा जिथे माहिती ठेवतात
string	अक्षरांचा मजकूर, अवतरण चिन्हात — "नमस्कार"
boolean	फक्त true (खरे) किंवा false (खोटे)
comparison	दोन गोष्टींची तुलना (>, <, ==)
condition (if)	"जर असे असेल तर तसे कर" असा निर्णय
loop	एखादे काम पुन्हा पुन्हा करणे
array	रांगेतले डबे — एका नावाखाली अनेक किमती
index	अरेतील डब्याचा क्रमांक (० पासून सुरू)
function	नाव दिलेले कामाचे जादूई मंत्र-पुस्तक
argument	फंक्शनला दिलेली माहिती
return	फंक्शनने परत दिलेले उत्तर
event	घडलेली घटना — क्लिक, माउस फिरवणे
event handler	घटना घडल्यावर चालणारा कोड
regular expression	अक्षरांचा नमुना ओळखण्याचा खास साचा
form	माहिती भरण्याचा अर्ज (नाव, ईमेल...)
validation	भरलेली माहिती बरोबर आहे का ते तपासणे
Date object	तारीख आणि वेळ सांभाळणारी वस्तू
timeout	ठरावीक वेळानंतर काम करायला लावणे



शाब्बास, कोडर मित्रा!



तुम्ही JavaScript चे सगळे ३९ धडे पूर्ण केलेत!

आता स्वतःची वेबसाइट बनवा, नवीन प्रयोग करा, आणि शिकत रहा.



बोट्ट म्हणतो: “चुका कारायला घाबरू नका — चुकांमधूनच प्रोग्रामर तयार होतात!”